



SILENT SENTINEL

a Motorola Solutions Company

External Universal Protocol specification

v2.4.7

- [1 Protocol versions](#)
- [2 Overview](#)
- [3 Messages format and addressing](#)
- [4 EUP messages](#)
 - [4.1 MotorControl messages](#)
 - [4.2 Camera module control messages](#)
 - [4.2.1 Lens control messages](#)
 - [4.2.2 Camera control messages](#)
 - [4.2.3 VideoSource control messages](#)
 - [4.2.4 VideoTracker control messages](#)
 - [4.2.5 VideoStabiliser control messages](#)
 - [4.2.6 MotionDetector control messages](#)
 - [4.2.7 ChangesDetector control messages](#)
 - [4.2.8 Classification control messages](#)
 - [4.2.9 DigitalZoom control messages](#)
 - [4.2.10 ImageFlip control messages](#)
 - [4.2.11 ColorFilter control messages](#)
 - [4.2.12 Clahe control messages](#)
 - [4.2.13 MotionMagnificator control messages](#)
 - [4.2.14 VideoStream control messages](#)
 - [4.2.15 Overlay control messages](#)
 - [4.2.16 Web RTC Overlay control messages](#)
 - [4.2.17 BadPixelProcessor control messages](#)
 - [4.3 Companion control messages](#)
 - [4.4 Lrf control messages](#)
 - [4.5 Speaker control messages](#)
 - [4.6 General control messages](#)

- [4.7 ProcedureManager control messages](#)
- [4.8 G5Laser control messages](#)
- [4.9 PeakBeam control messages](#)
- [4.10 ExternalUp control messages](#)
- [4.11 PelcoD control messages](#)
- [4.12 Combined commands](#)
- [5 Example of control program](#)
- [6 Example of detection results subscription](#)
- [7 JSON based communication](#)

1 Protocol versions

Table 1.1 - Protocol versions.

Version	Release date	What's new
1.0.0	02.02.2024	First version.
1.1.0	04.02.2024	- Update command list.
1.2.0	11.02.2024	- Update addressing. - Update command list.
1.2.1	12.02.2024	- Update Camera1(2,3,4) commands list. - Add general system commands.
1.2.2	13.02.2024	- Update Speaker commands list. - Add response time information. - Add subscription mechanism for detection results.
1.3.0	17.02.2024	- Add JSON-based messages support.
1.3.1	20.03.2025	- Low latency UDP ports control has been changed.
1.4.0	02.04.2025	- Add ProcedureManager commands.
1.4.1	10.06.2025	- Update ProcedureManager commands.
1.4.2	21.06.2025	- Add new lens commands and camera specific commands.
1.4.3	23.06.2025	- Add new General commands.
1.4.4	09.07.2025	- Add combined commands format.
1.4.5	15.07.2025	- Add new General commands. - Add new MotorControl commands.
1.4.6	20.07.2025	- Update MotorControl commands.
1.4.7	04.08.2025	- Update General commands.

Version	Release date	What's new
1.4.8	20.08.2025	- Update MotorControl commands.
1.4.9	20.10.2025	- Add new commands for WebRTC overlay control.
1.4.10	22.10.2025	- Add new commands for RTP stream control.
2.0.0	05.11.2025	- Message format changed - added obligatory to put '\n' symbol at the end of command.
2.0.1	28.11.2025	- Add new commands for RTMP, HLS and SRT servers.
2.1.0	15.12.2025	- Update command list. Update document structure.
2.2.0	30.01.2026	- Add LaserIlluminator and PeakBeam commands. - Add new functional modules for PelcoD and External UP communication.
2.3.0	04.02.2026	- Add new submodule "BadPixelProcessor" for Camera1(2,3,4) functional modules. - Extended video tracker commands.
2.3.1	09.02.2026	- Fixed "General" commands mistakes.
2.3.2	13.02.2026	- Add new params for VideoStream1/2 control.
2.4.0	01.03.2026	- Removed video streaming commands from "General" section. - Added new commands to "VideoStream" section.
2.4.1	06.03.2026	- Updated commands description.
2.4.2	11.03.2026	- Updated commands description.
2.4.3	16.03.2026	- Add new MotorControl Homing Delay commands.
2.4.4	17.03.2026	- Add new Camera commands.
2.4.5	16.04.2026	- Fixed commands description mistakes.
2.4.6	17.04.2026	- Fixed MotorControl commands description.
2.4.7	13.05.2026	- Updated the description of VideoTracker commands.

2 Overview

The External Control Universal Protocol (**EUP**) is designed to allow external systems to control the **NexOS** platform and receive data from it. **EUP** uses UDP or TCP communication, where **NexOS** listens on a specific UDP / TCP port (port **34030** by default for UDP and **34030** for TCP) and responds to incoming messages. Main principles and features:

1. **String-Based Protocol:** **EUP** is a string-based protocol, meaning messages can be read by humans without additional preprocessing. **WARNING:** Each message starts with **ID:** and ends with end of line symbol '\n'.

2. **Request-Response Principle:** The protocol operates on a request-response basis, meaning **NexOS** does not send messages to external users on its own but only responds to incoming messages. The **NexOS** has an option for notification of clients about detection results (detected objects from motion detector, changes detector or classification). To receive notifications the client should subscribe to it.
3. **Flexible Message Routing: EUP** can route messages to various services based on address, supporting different system configurations, such as:
 - A single main board with all services
 - A main board with separate payload boards for each camera
 - A main board with payload boards for multiple cameras
 - Configurations including main board, camera payload board, and non-optical payload boards, etc.
4. **Supported Message Types:**
 - **Command:** Used to trigger actions or modify parameters of any functional module.
 - **Query:** Requests the value of a parameter from any functional module.
 - **Reply:** Sent by **NexOS** in response to a **Query**, including the requested parameter's value.
 - **Ack:** Sent by **NexOS** to acknowledge that an action command or parameter change has been executed or accepted.
 - **Nac:** Sent by **NexOS** when a command is invalid or unaccepted.
5. **Multi-Client Support: EUP** allows multiple clients to control the system simultaneously. The **NexOS** recognizes client's IP and port and sends replies to the appropriate client. A client can initialize any UDP port to communicate with **NexOS**.
6. **Message IDs:** Each client message and **NexOS** reply includes a unique ID for tracking purposes.
7. The protocol optionally supports JSON-based communication. The **NexOS** automatically detects the type of message (string-based or JSON-based) and replies in the appropriate format.

3 Messages format and addressing

General Message Format

The **EUP** messages follow a structured format:

```
<ID:N>/<MessageType>/<Module>/<Submodule (if necessary)>/<Parameter or Action Command>
```

- [ID:N](#) is the unique identifier for each message, where **N** is the message ID. This ID is used by the system to match replies with specific client requests, ensuring that each response returns to the appropriate client.
- is message type key word, one from list: **Command, Query, Reply, Ack, Nac**.
- defines the target module (device). All modules can be found in **Table 3.2**.
- specifies the structure, that command or parameter setting refers to. All Submodules can be found in **Table 3.3**.
- is the chosen parameter (with a value following a ':' character) or an action command name with or without parameter. Specific for each submodule.

Table 3.1 - Types of messages (all **EUP** messages start with this keywords after [ID:N](#)):

Message Type	Description
Command	Sent from the Client to NexOS . Used to execute an action or change a parameter in any module.
Query	Sent from the Client to NexOS . Used to request the value of a parameter of particular module.
Reply	Sent from NexOS to the Client . Response to a valid Query message, including the requested parameter value.
Ack	Sent from NexOS to the Client . Response to a valid Command message, indicating the command was executed or accepted.
Nac	Sent from NexOS to the Client . Response to an invalid Command or Query message.

Table 3.2 - Modules (field of **EUP** messages) supported by **NexOS**.

Module	Description
MotorControl	Motor control module.
Camera1	First camera (usually thermal).
Camera2	Second camera (usually daylight).
Camera3	Third camera (usually SWIR).
Camera4	Fourth camera.
CompanionMain	Companion board module which is connected to the main board inside pan-tilt unit.
Companion1	Companion board module which is connected to first payload board.
Companion2	Companion board module which is connected to second payload board.
Companion3	Companion board module which is connected to third payload board.
General	General system parameters.
ExternalUp	External Universal Protocol control parameters.
PelcoD	External PelcoD control parameters.
G5Laser	Laser illuminator.
PeakBeam	Peak beam.
Lrf	Laser range finder service.
Speaker	Speaker service.

Table 3.3 - Submodules of **Camera1(2,3,4)** module. In addition to common parameters.

Submodule	Description
Lens	Lens submodule.
Camera	Camera submodule.
VideoSource	Video source submodule.
VideoTracker	Video tracker submodule.
VideoStabiliser	Video stabiliser submodule.
MotionDetector	Motion detector submodule.
ChangesDetector	Changes detector submodule.
Classification	Classification submodule.
DigitalZoom	Digital zoom submodule.
Clahe	CLAHE (defog) submodule.
ImageFlip	Image flip submodule.
ColorFilter	Color filter submodule.
MotionMagnificator	Motion magnification.
VideoStream1	First video stream (RTSP/RTMP/HLS/SRT).
VideoStream2	Second video stream (RTSP/RTMP/HLS/SRT).
Overlay	Video overlay.
WebRtcOverlay	WebRTC video overlay.
BadPixelProcessor	Bad pixel detection and compensation.

Examples of **EUP** Messages:

Message	Explanation
ID:0/Command/Camera1/Lens/ZoomIn	Command to move zoom in (tele) for camera 1.
ID:1/Command/Camera2/Camera/Profile:2	Command to set setting profile 2 to camera core of camera 2.
ID:2/Query/CompanionMain/PowerChannel1Temperature	Query temperature of first power channel of companion board which is connected to main board.
ID:2/Reply/CompanionMain/PowerChannel1Temperature:50	Reply about temperature of first power channel of companion board which is connected to main board.

Message	Explanation
ID:3/Command/Camera1/VideoStabiliser/On	Action command to enable video stabilizer in camera 1.
ID:4/Command/Camera1/VideoTracker/Reset	Action command to reset video tracker in camera 1.

4 EUP messages

4 1 MotorControl messages

MotorControl module is responsible for pan-tilt unit movement. Motor control service is running on main board.

Table 4.1.1 - Action commands and commands to change parameters. Messages start from **.../Command/MotorControl/...** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**. Central system service sends command to **MotorControl** service and replies **Ack** if the message was sent without checking if the message is valid.

Command	Description
Pan/Left/AtRate:n	Move PT Left. Parameter n : speed [0-100%] float value of max pan velocity Pan/MaxSpeed .
Pan/Right/AtRate:n	Move PT Right. Parameter n : speed [0-100%] float value of max pan velocity Pan/MaxSpeed .
Pan/Left/AtFineRate:n	Move PT Left. Parameter n : speed [0-65535] float value mapped to pan velocity Pan/MaxSpeed .
Pan/Right/AtFineRate:n	Move PT Right. Parameter n : speed [0-65535] float value mapped to pan velocity Pan/MaxSpeed .
Pan/MaxSpeed:n	Set maximum Pan rotation speed. Parameter n : speed [deg/s] only positive float value. If the value is bigger than characteristics of the Pan-Tilt unit, the system will set speed to maximum allowed value.
Pan/PositionSpeed:n	Set Pan rotation speed for ToPos command. Parameter n : speed [deg/s] only positive float value. If the value is bigger than characteristics of the Pan-Tilt unit, the system will set speed to maximum allowed value.
Pan/Left/AtDps:n	Move PT Left with speed. Parameter n : speed [deg/s] float signed value. The absolute value must be <= maximum speed Pan/MaxSpeed .
Pan/Right/AtDps:n	Move PT Right with speed. Parameter n : speed [deg/s] float signed value. The absolute value must be <= maximum speed Pan/MaxSpeed .
Pan/Stop	Stop Pan movement.
Pan/ToPos:n	Go to Pan position. Parameter n : position [deg] only positive float value from 0.0 to 360.0.

Command	Description
Pan/SetZero	Set user zero point at current Pan position.
Pan/ResetZero	Set user zero point to default zero for Pan.
Pan/ZeroPos:n	Set user zero Pan position. Parameter n : position [deg] only positive float value from 0.0 to 360.0.
Pan/AddPos:n	NO PASSED Adjust Pan position Parameter n : position [deg] signed float value from -180.0 to 180.0.
Pan/InvertMovement:s	Set invert Pan movement. When On , swaps the direction of Pan/Left and Pan/Right rate/speed commands. Does not affect position commands such as Pan/ToPos . Parameter s : Off or On .
Pan/LeftLimit:n	Set Pan Left limit. Parameter n : position [deg] only positive float value from 0.0 to 360.0.
Pan/RightLimit:n	Set Pan Right limit. Parameter n : position [deg] only positive float value from 0.0 to 360.0.
Tilt/Up/AtRate:n	Move PT Up. Parameter n : speed [0-100%] float value of max tilt velocity Tilt/MaxSpeed .
Tilt/Down/AtRate:n	Move PT Down. Parameter n : speed [0-100%] float value of max tilt velocity Tilt/MaxSpeed .
Tilt/Up/AtFineRate:n	Move PT Up. Parameter n : speed [0-65535] float value mapped to tilt velocity Tilt/MaxSpeed .
Tilt/Down/AtFineRate:n	Move PT Down. Parameter n : speed [0-65535] float value mapped to tilt velocity Tilt/MaxSpeed .
Tilt/MaxSpeed:n	Set maximum Tilt rotation speed. Parameter n : speed [deg/s] only positive float value. If the value is bigger than characteristics of the Pan-Tilt unit, the system will set speed to maximum allowed value.
Tilt/PositionSpeed:n	Set Tilt rotation speed for ToPos commands. Parameter n : speed [deg/s] only positive float value. If the value is bigger than characteristics of the Pan-Tilt unit, the system will set speed to maximum allowed value.
Tilt/Up/AtDps:n	Move PT Up with speed. Parameter n : speed [deg/s] float signed value. The absolute value must be <= maximum speed Tilt/MaxSpeed .
Tilt/Down/AtDps:n	Move PT Down with speed. Parameter n : speed [deg/s] float signed value. The absolute value must be <= maximum speed Tilt/MaxSpeed .
Tilt/Stop	Stop Tilt movement.
Tilt/ToPos:n	Go to Tilt position. Parameter n : position [deg] float value from -90.0 to 90.0.
Tilt/SetZero	Set user zero point at current Tilt position.
Tilt/ResetZero	Set user zero point to default zero for Tilt.

Command	Description
Tilt/ZeroPos:n	Set user zero Tilt position. Parameter n : position [deg] float value from -90.0 to 90.0.
Tilt/AddPos:n	Adjust Tilt position Parameter n : position [deg] signed float value from -90.0 to 90.0.
Tilt/InvertMovement:s	Set invert Tilt movement. When On , swaps the direction of Tilt/Up and Tilt/Down rate/speed commands. Does not affect position commands such as Tilt/ToPos . Parameter s : Off or On .
Tilt/UpLimit:n	Set Tilt Up limit. Parameter n : position [deg] float value from -90.0 to 90.0.
Tilt/DownLimit:n	Set Tilt Down limit. Parameter n : position [deg] float value from -90.0 to 90.0.
BlockPT:s	Suppresses incoming PT movement commands. Parameter s : Off or On .
ResetController	Reset the controller module.
ZoomDependentMode:n	Set Zoom dependent mode. Control speed adjusted to Zoom position of chosen camera. Parameter n : 0 - Off 1 - Master camera 1 2 - Master camera 2 3 - Master camera 3
ZoomDependentFactor:n	Set Zoom dependent factor. Defines the reduction coefficient for control speed. At maximum zoom, control speed is divided by $(N + 1)$, where N is the value of this parameter. Note : it is not recommended to change default values.
AccVstart:n	Set starting velocity (VS) used in the acceleration/deceleration profile for both Pan and Tilt axes. Parameter n : float velocity [deg/s], non-negative. The effective per-axis value is bounded by that axis MaxSpeed. Applied to both Pan and Tilt simultaneously. Note : it is not recommended to change default values.
AccInitialAcc:n	Set initial acceleration (AI) for both Pan and Tilt axes. Parameter n : float acceleration [deg/s ²], non-negative. Applied to both Pan and Tilt simultaneously. Note : it is not recommended to change default values.
AccTargetVelocity:n	Set initial target velocity (VI) for both Pan and Tilt axes. Parameter n : float velocity [deg/s], non-negative. The effective per-axis value is bounded by that axis MaxSpeed. Applied to both Pan and Tilt simultaneously. Note : it is not recommended to change default values.
AccAccMax:n	Set maximum acceleration (AM) for both Pan and Tilt axes. Parameter n : float acceleration [deg/s ²], non-negative. Applied to both Pan and Tilt simultaneously. Note : it is not recommended to change default values.

Command	Description
AccVelMax:n	Set maximum velocity (VM) for both Pan and Tilt axes. Parameter n: float velocity [deg/s], non-negative. Applied to both Pan and Tilt simultaneously. Note: it is not recommended to change default values.
AccDecMax:n	Set maximum deceleration (DM) for both Pan and Tilt axes. Parameter n: float deceleration [deg/s ²], non-negative. Applied to both Pan and Tilt simultaneously. Note: it is not recommended to change default values.
AccDecRate:n	Set deceleration for stopping (DS) for both Pan and Tilt axes. Parameter n: float deceleration [deg/s ²], non-negative. Applied to both Pan and Tilt simultaneously. Note: it is not recommended to change default values.
AccDecVstop:n	Set release/stop velocity (VR) — velocity at which motion is considered stopped — for both Pan and Tilt axes. Parameter n: float velocity [deg/s], non-negative. Applied to both Pan and Tilt simultaneously. Note: it is not recommended to change default values.
HomingDelay:n	Set homing delay mode. This option delays homing procedure for the duration specified with HomingDelayTime command. Parameter n: 0 - Off 1 - On
HomingDelayTime:n	Set homing delay time. Parameter n: time in seconds, value from 0.0 to 3600.0

Table 4.1.2 - Query commands. Messages start from **.../Query/MotorControl/...** Response from system starts with **.../Reply/MotorControl/...** Central control service sends message to **MotorControl** service. Expected response time: **IP ping time to the system + 5 msec** if central control service has received a valid reply from **MotorControl** service, or **IP ping time to the system + 105 msec** if no valid reply from **MotorControl** service.

Command	Description
Pan/Pos	Query Pan position. Response: .../Pan/Pos:n . Parameter n: position [deg] from 0.0 to 360.0.
Pan/MaxSpeed	Query maximum Pan speed. Response: .../Pan/MaxSpeed:n . Parameter n: maximum Pan speed [deg/s].
Pan/PositionSpeed	Query Pan speed for ToPos commands. Response: .../Pan/PositionSpeed:n . Parameter n: maximum Pan speed [deg/s].
Pan/ZeroPos	Query user Pan zero position. Response: .../Pan/ZeroPos:n . Parameter n: position [deg] from 0.0 to 360.0.
Pan/Invert	Query invert Pan position mode (telemetry only). Response: .../Pan/Invert:s . Parameter s: Off or On .
Pan/InvertMovement	Query invert Pan control mode. Parameter s: Off or On .

Command	Description
Pan/LeftLimit	Query Pan Left limit position. Response: .../Pan/LeftLimit:n . Parameter n : limit position [deg] from 0.0 to 360.0.
Pan/RightLimit	Query user Right limit position. Response: .../Pan/RightLimit:n . Parameter n : limit position [deg] from 0.0 to 360.0.
Tilt/Pos	Query Tilt position. Response: .../Tilt/Pos:n . Parameter n : position [deg] from -90.0 to 90.0.
Tilt/MaxSpeed	Query maximum Tilt speed. Response: .../Tilt/MaxSpeed:n . Parameter n : maximum Tilt speed [deg/s].
Tilt/PositionSpeed	Query Tilt speed for ToPos commands. Response: .../Tilt/PositionSpeed:n . Parameter n : maximum Tilt speed [deg/s].
Tilt/ZeroPos	Query user Tilt zero position. Response: .../Tilt/ZeroPos:n . Parameter n : position [deg] from -90.0 to 90.0.
Tilt/Invert	Query invert Tilt position mode (telemetry only). Response: .../Tilt/Invert:s . Parameter s : Off or On .
Tilt/InvertMovement	Query invert Tilt control mode. Parameter s : Off or On .
Tilt/UpLimit	Query Tilt Up limit position. Response: .../Tilt/UpLimit:n . Parameter n : limit position [deg] from -90.0 to 90.0.
Tilt/DownLimit	Query Tilt Down limit position. Response: .../Tilt/DownLimit:n . Parameter n : limit position [deg] from -90.0 to 90.0.
BlockPT	Query suppresses mode for incoming PT movement commands. Response: .../BlockPt:s . Parameter s : Off or On .
IsMoving	Query movement status. Response: .../IsMoving:s . Parameter s : 1 (if moving) or 0 (if not moving).
ZoomDependentMode	Query Zoom dependent mode. Control speed adjusted to Zoom position of chosen camera. Parameter n : 0 - Off 1 - Master camera 1 2 - Master camera 2 3 - Master camera 3
ZoomDependentFactor	Query Zoom dependent factor. Defines the reduction coefficient for control speed. At maximum zoom, control speed is divided by $(N + 1)$, where N is the value of this parameter.
AccVstart	Query starting velocity (VS) used in the acceleration/deceleration profile for both Pan and Tilt axes. Response: .../AccVstart:s . Parameter n : float velocity [deg/s], non-negative. The effective per-axis value is bounded by that axis MaxSpeed. Applied to both Pan and Tilt simultaneously.

Command	Description
AccInitialAcc	Query initial acceleration (AI) for both Pan and Tilt axes. Response: .../AccInitialAcc:s . Parameter n: float acceleration [deg/s ²], non-negative. Applied to both Pan and Tilt simultaneously.
AccTargetVelocity	Query initial target velocity (VI) for both Pan and Tilt axes. Response: .../AccTargetVelocity:s . Parameter n: float velocity [deg/s], non-negative. The effective per-axis value is bounded by that axis MaxSpeed. Applied to both Pan and Tilt simultaneously.
AccAccMax	Query maximum acceleration (AM) for both Pan and Tilt axes. Response: .../AccAccMax:s . Parameter n: float acceleration [deg/s ²], non-negative. Applied to both Pan and Tilt simultaneously.
AccVelMax	Query maximum velocity (VM) for both Pan and Tilt axes. Response: .../AccVelMax:s . Parameter n: float velocity [deg/s], non-negative. Applied to both Pan and Tilt simultaneously.
AccDecMax	Query maximum deceleration (DM) for both Pan and Tilt axes. Response: .../AccDecMax:s . Parameter n: float deceleration [deg/s ²], non-negative. Applied to both Pan and Tilt simultaneously.
AccDecRate	Query deceleration for stopping (DS) for both Pan and Tilt axes. Response: .../AccDecRate:s . Parameter n: float deceleration [deg/s ²], non-negative. Applied to both Pan and Tilt simultaneously.
AccDecVstop	Query release/stop velocity (VR) — velocity at which motion is considered stopped — for both Pan and Tilt axes. Response: .../AccDecVstop:s . Parameter n: float velocity [deg/s], non-negative. Applied to both Pan and Tilt simultaneously.
HomingDelay	Query homing delay status. Response: .../HomingDelay:n . Parameter n: 0 - Off 1 - On
HomingDelayTime	Query delay time before homing procedure. Parameter n: time in seconds, value from 0.0 to 3600.0
HomingDelayRemainingTime	Query remaining time to start homing procedure. Parameter n: time in seconds, value from 0.0 to 3600.0.

Some example messages:

[client -> system] **ID:10/Command/MotorControl/Pan/ToPos:120.5** - go to pan position 120.5 degree.

[system -> client] **ID:10/Ack** - command accepted.

[client -> system] **ID:11/Command/MotorControl/Pan/ToPos:500** - go to pan position 500 degree.

[system -> client] **ID:11/Nac** - command not accepted.

[client -> system] **ID:12/Query/MotorControl/Tilt/DownLimit** - Query tilt down limit.

[system -> client] **ID:12/Reply/MotorControl/Tilt/DownLimit:-60.1** - tilt down limit -60 degree.

[client -> system] **ID:13/Command/MotorControl/Pan/RightLimit:10.8** - Set right pan limit to 10.8 degree.

[system -> client] **ID:13/Ack** - command accepted.

4 2 Camera module control messages

Camera1, **Camera2** and **Camera3** modules are responsible for camera control. Each camera module includes several submodules.

Table 4.2.1 - Action commands on module level. Messages start from **.../Command/Camera1(2,3,4)/...**. Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Reset	Restart Jafar service for the camera module.
Restart	Restart Jafar service for the camera module. Alias for Reset .

4 2 1 Lens control messages

Lens is a submodule which is part of the **Camera1**, **Camera2**, and **Camera3** modules. This submodule is responsible for lens control.

Table 4.2.1.1 - Action Commands and Parameter Change Commands: Messages begin with **.../Command/Camera1(2,3,4)/Lens/...**. If the command is successfully sent to the lens controller, the system responds with **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
ZoomIn	Move zoom tele (in).
ZoomOut	Move zoom wide (out).
ZoomToPos:n	Move zoom to position. Parameter n : integer position value from 0 (full wide) to 65535 (full tele).
ZoomPos:n	Move zoom to position. Parameter n : integer position value from 0 (full wide) to 65535 (full tele).
ZoomStop	Stop zoom movement including stop zoom to position command.
FocusFar	Move focus far.
FocusNear	Move focus near.
FocusToPos:n	Move focus to position. Parameter n : integer position value from 0 (full near) to 65535 (full far).
FocusPos:n	Move focus to position. Parameter n : integer position value from 0 (full near) to 65535 (full far).
FocusStop	Stop focus movement including stop focus to position command.
IrisOpen	Move iris to open position. Not available for thermal lenses.
IrisClose	Move iris to close position. Not available for thermal lenses.
IrisToPos:n	Move iris to position. Parameter n : integer position value from 0 (full close) to 65535 (full open). Not available for thermal lenses.

Command	Description
IrisPos:n	Move iris to position. Parameter n : integer position value from 0 (full close) to 65535 (full open). Not available for thermal lenses.
IrisHwPos:n	Move iris to hardware position (specific for each lens). Parameter n : hardware iris position specific for each lens.
IrisStop	Stop iris movement including stop iris to position command. Not available for thermal lenses.
RunAf	Start autofocus.
StopAf	Stop autofocus.
Restart	Restart lens controller.
FocusMode:n	Set focus mode. Parameter n (integer): 0 - Manual focus control, 1 - Auto focus control (continuous auto focus).
FilterMode:n	Set filter mode. Parameter n (integer): 0 - Filter on, 1 - Filter off. Parameter value depends on lens controller.
AfRoiMode:n	Set auto focus ROI mode. Parameter n (integer): 0 - Auto detection, 1 - Center 30%, 2 - Center 50%, 3 - Center 70%.
ZoomSpeed:n	Set zoom speed. Parameter n : integer speed value [0-100%] of max hardware zoom speed.
FocusSpeed:n	Set focus speed. Parameter n : integer speed value [0-100%] of max hardware focus speed.
IrisSpeed:n	Set iris speed. Parameter n : integer speed value from [0-100%] of max hardware iris speed.
ZoomMaxSpeed:n	Set maximum relative zoom speed used as an upper limit for Zoom ... AtRate commands. Parameter n (integer): relative speed [0-100%].
FocusMaxSpeed:n	Set maximum relative focus speed used as an upper limit for Focus ... AtRate commands. Parameter n (integer): relative speed [0-100%].
FocusFactorThreshold:n	Set threshold for changes of focus factor to start refocus. Parameter n : integer value [0-100%], 100% - changing x2.
RefocusTimeoutSec:n	Set timeout for automatic refocus. Parameter n : seconds from 0 - no automatic refocus to 100000 - maximum value.
IrisMode:n	Set iris mode. Parameter n (integer): 0 - manual iris control, 1 - auto iris control. Not available for thermal lenses.
ExtenderMode:n	Set extender mode. Parameter n (integer): 0 - no extender, 1 - x2 extender. If the lens supports extender.
StabiliserMode:n	Set stabilization mode. Parameter n (integer): 0 - no stabilization, 1 - stabilization. If lens supports stabilization.

Command	Description
AfRange:n	Set autofocus range. Parameter n (integer): auto focus range. Value depends on lens controller.
XFovDeg:n	Set horizontal field of view angle, degree. Parameter n (float): horizontal field of view angle [deg] in range [0:180].
YFovDeg:n	Set vertical field of view angle, degree. Parameter n (float): vertical field of view angle [deg] in range [0:180].
Custom1:n	Set custom lens parameter. Parameter n (float / integer) depends on lens controller.
Custom2:n	Set custom lens parameter. Parameter n (float / integer) depends on lens controller.
Custom3:n	Set custom lens parameter. Parameter n (float / integer) depends on lens controller.

Table 4.2.1.2 - Lens submodule queries. Messages start from **.../Query/Camera1(2,3,4)/Lens/....** Response from system starts with **.../Reply/Camera1(2,3,4)/Lens/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
ZoomPos	Query zoom position. Response: .../ZoomPos:n . Parameter n : integer position value from 0 (full wide) to 65535 (full tele).
FocusPos	Query focus position. Response: .../FocusPos:n . Parameter n : position value from 0 (full near) to 65535 (full far).
IrisPos	Query iris position. Response: .../IrisPos:n . Parameter n : position value from 0 (full close) to 65535 (full open).
ZoomSpeed	Query zoom speed. Response: .../ZoomSpeed:n . Parameter n : speed value [0-100%] (relative for particular lens).
FocusSpeed	Query focus speed. Response: .../FocusSpeed:n . Parameter n : speed value [0-100%] (relative for particular lens).
IrisSpeed	Query iris speed. Response: .../IrisSpeed:n . Parameter n : speed value [0-100%] (relative for particular lens).
FocusMode	Query focus mode. Response: .../FocusMode:n . Parameter n (integer): 0 - Manual focus control, 1 - Auto focus control (continuous auto focus).
FilterMode	Query filter mode. Response: .../FilterMode:n . Parameter n : depends on lens controller implementation but it is recommended to keep default values: 0 - Filter on, 1 - Filter off.
AfRoiMode	Query auto focus ROI mode. Response: .../AfRoiMode:n . Parameter n (integer): 0 - Auto detection, 1 - Center 30%, 2 - Center 50%, 3 - Center 70%.

Command	Description
IsConnected	Query connection status. Connection status shows if lens controller has communication with lens hardware. Response: .../IsConnected:n . Parameter n : 0 - no response from lens, 1 - connected.
IsOpen	Query lens controller initialization status. Response: .../IsOpen:n . Parameter n (integer): 0 - lens controller not init, 1 - init.
FocusFactor	Query focus factor value (only for lens controllers which support custom auto focus). Response: .../FocusFactor:n . Parameter n - focus factor in relative units.
FocusFactorThreshold	Query focus factor threshold for changes of focus factor to start refocus (only for lens controllers which support custom auto focus). Response: .../FocusFactorThreshold:n . Parameter n : 0% - no check, 100% - changing x2.
RefocusTimeoutSec	Query focus timeout for automatic refocus in seconds (only for lens controllers which support custom auto focus). Response: .../RefocusTimeoutSec:n . Parameter n - refocus timeout, sec. 0 - disabled.
IsAfActive	Query active auto focus status. Response: .../AfIsActive:n . Parameter n (integer): 0 - auto focus procedure not active, 1 - lens controller doing autofocus.
IrisMode	Query iris mode. Response: .../IrisMode:n . Parameter n default (integer): 0 - manual iris control, 1 - auto iris control.
ExtenderMode	Query extender mode. Response: .../ExtenderMode:n . Parameter n (integer): 0 - no extender, 1 - x2 extender.
AfRange	Query auto focus range. Response: .../AfRange:n . Parameter n depends on lens controller.
XFovDeg	Query current horizontal field of view angle, degree. Response: .../XFovDeg:n . Parameter n : float horizontal field of view angle [deg].
YFovDeg	Query current vertical field of view angle, degree. Response: .../YFovDeg:n . Parameter n : float vertical field of view angle [deg].
Temperature	Query lens temperature. Response: .../Temperature:n . Parameter n : float lens temperature value, [deg] Celsius.
Custom1	Query lens custom 1 parameter. Response: .../Custom1:n . Parameter n depends on lens controller.
Custom2	Query lens custom 2 parameter. Response: .../Custom2:n . Parameter n depends on lens controller.
Custom3	Query lens custom 3 parameter. Response: .../Custom3:n . Parameter n depends on lens controller.

Some example messages:

[client -> system] **ID:14/Command/Camera1/Lens/ZoomToPos:12000** - Go to zoom position 12000.

[system -> client] **ID:14/Ack** - Command accepted.

[client -> system] **ID:15/Command/Camera1/Lens/AfRoiMode:0** - Set auto focus ROI to auto detection mode.

[system -> client] **ID:15/Ack** - Command accepted.

[client -> system] **ID:16/Query/Camera2/Lens/FocusPos** - Query focus position from camera 2.

[system -> client] **ID:16/Reply/Camera2/Lens/FocusPos:100** - Focus position from camera 2.

[client -> system] **ID:17/Query/Camera3/Lens/IsAfActive** - Query auto focus status.

[system -> client] **ID:17/Reply/Camera3/Lens/IsAfActive:0** - Auto focus not active.

4 2 2 Camera control messages

Camera is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **Camera** submodule is responsible for camera core (image sensor) control.

Table 4.2.2.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/Camera/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Restart	Restart camera controller.
Nuc	Do NUC (Flat Field Correction). Only for thermal and SWIR cameras.
ApplyParams	Apply params.
SaveParams	Save params in camera memory.
MenuOn	Enter camera menu. Some cameras don't support onscreen menu.
MenuOff	Exit camera menu. Some cameras don't support onscreen menu.
MenuSet	Menu set. Some cameras don't support onscreen menu.
MenuUp	Menu navigation up. Some cameras don't support onscreen menu.
MenuDown	Menu navigation down. Some cameras don't support onscreen menu.
MenuLeft	Menu navigation left. Some cameras don't support onscreen menu.
MenuRight	Menu navigation right. Some cameras don't support onscreen menu.
FreezeSec:n	Freeze image for n seconds. 0 - freeze without timeout (integer).
Defreeze	Disable freeze.

Command	Description
DisplayMode:n	Set display menu mode. Parameter n (integer): 0 - Off, 1 - On. If display menu mode 1 the camera will enter menu if supported.
ExposureMode:n	Set exposure mode. Parameter n (integer): 0 - Manual, 1 - Auto (default), 2 - Shutter priority, 3 - Aperture priority.
ExposureTime:n	Set exposure time of the camera sensor. Parameter n (integer): exposure time which is limited by the frame interval. Camera controller interprets the values as 100µs per unit, where the value 1 stands for 1/10000th of a second, 10000 for 1 second and 100000 for 10 seconds.
WhiteBalanceMode:n	Set white balance mode. Parameter n (integer): 0 - Manual, 1 - Auto.
WhiteBalanceArea:n	Set white balance area. Parameter n (integer): white balance area. Value depends on camera controller.
WideDynamicRangeMode:n	Set wide dynamic range mode. Parameter n (integer): 0 - Off, 1 - On.
StabilisationMode:n	Set image stabilization mode. Parameter n inside camera core (integer): 0 - Off, 1 - On.
IsoSensitivity:n	Set ISO sensitivity. Parameter n (integer): sensitivity. Value depends on camera controller.
SceneMode:n	Set scene mode. Parameter n (integer): scene mode. Value depends on camera controller.
BrightnessMode:n	Set brightness mode. Parameter n (integer): 0 - Manual, 1 - Auto.
Brightness:n	Set brightness level. Parameter n (integer): brightness level [0-100%].
Contrast:n	Set contrast level. Parameter n (integer): contrast level [0-100%].
GainMode:n	Set gain mode. Parameter n (integer): 0 - Manual, 1 - Auto.
Gain:n	Set gain level. Parameter n (integer): gain level [0-100%].
SharpeningMode:n	Set sharpening mode. Parameter n (integer): 0 - Manual, 1 - Auto.
Sharpening:n	Set sharpening level. Parameter n (integer): [0-100%].
Palette:n	Set palette. Parameter n (integer): palette mode. Value depends on camera controller. Default values: 0 - White hot, 1 - Black hot.
AgcMode:n	Set analog gain control mode. Parameter n (integer): 0 - Manual, 1 - Auto.
ShutterMode:n	Set shutter mode. Parameter n (integer): 0 - Manual, 1 - Auto.
ShutterPos:n	Set shutter position. Parameter n (integer): 0 (full close) - 65535 (full open).

Command	Description
ShutterSpeed:n	Set shutter speed n : 0 - 100% .
ExposureCompensationMode:n	Set exposure compensation mode. Parameter n (integer): 0 - Off, 1 - On.
ExposureCompensationPos:n	Set exposure compensation position. Parameter n (integer): exposure compensation position.
DefogMode:n	Set defog mode. Parameter n (integer): 0 - Off, 1 - On.
DehazeMode:n	Set dehaze mode. Parameter n (integer): 0 - Off, 1 - On. Particular cameras can have specific values.
NoiseReductionMode:n	Set noise reduction mode. Parameter n (integer): 0 - Off, 1 - 2D, 3 - 3D.
BlackAndWhiteFilterMode:n	Set black and white filter mode. Parameter n (integer): 0 - Off, 1 - On.
FilterMode:n	Set filter mode. Parameter n (integer): filter mode. Value depends on camera controller.
NucMode:n	Set NUC mode for thermal cameras. Parameter n (integer): 0 - Manual, 1 - Auto.
AutoNucIntervalSec:n	Set auto NUC interval for thermal cameras. Parameter n (integer): nuc interval [sec] 0 (Off) - 100000 .
AutoNucIntervalMin:n	Set auto NUC interval for thermal cameras. Parameter n (integer): nuc interval [minutes] 0 (Off) - 100000 .
DdeMode:n	Set DDE mode. Parameter n (integer): 0 - Off, 1 - On.
DdeLevel:n	Set DDE level. Parameter n (integer): DDE level. Value depends on camera controller.
AlcGate:n	Set ALC gate n . Value depends on camera controller.
Sensitivity:n	Set sensor sensitivity. Parameter n (integer): sensitivity. Value depends on camera controller.
ChangingMode:n	Set changing mode (day / night). Parameter n (integer): 0 - Manual, 1 - Auto.
ChangingLevel:n	Set changing level (day / night). Parameter n (integer): changing level. Value depends on camera controller.
ChromaLevel:n	Set chroma level. Parameter n (integer): chroma level [0-100%].
Detail:n	Set details enhancement. Parameter n (integer): detail enhancement level [0-100%].
Profile:n	Set camera settings profile. Parameter n (integer): profile. Value depends on camera controller.

Command	Description
Custom1:n	Set camera custom parameter. Parameter n (float / integer) depends on camera controller.
Custom2:n	Set camera custom parameter. Parameter n (float / integer) depends on camera controller.
Custom3:n	Set camera custom parameter. Parameter n (float / integer) depends on camera controller.
AgcRoiX0:n	Set AGC ROI left horizontal coordinate. Parameter n (integer): x0 position, pixels (original camera resolution).
AgcRoiY0:n	Set AGC ROI top vertical coordinate. Parameter n (integer): y0 position, pixels (original camera resolution).
AgcRoiX1:n	Set AGC ROI right horizontal coordinate. Parameter n (integer): x1 position, pixels (original camera resolution).
AgcRoiY1:n	Set AGC ROI bottom vertical coordinate. Parameter n (integer): y1 position, pixels (original camera resolution).

Table 4.2.2.2 - Camera submodule queries. Messages start from **.../Query/Camera1(2,3,4)/Camera/....** Response from system starts with **.../Reply/Camera1(2,3,4)/Camera/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
DisplayMode	Query display menu mode. Response: .../DisplayMode:n . Parameter n (integer): 0 - Off, 1 - On.
ExposureMode	Query exposure mode. Response: .../ExposureMode:n . Parameter n (integer): 0 - Manual, 1 - Auto (default), 2 - Shutter priority, 3 - Aperture priority.
ExposureTime	Query exposure time of the camera sensor. Response: .../ExposureTime:n . The exposure time is limited by the frame interval. Camera controller interprets the values as 100µs per unit, where the value 1 stands for 1/10000th of a second, 10000 for 1 second and 100000 for 10 seconds.
WhiteBalanceMode	Query white balance mode. Response: .../WhiteBalanceMode:n . Parameter n (integer): 0 - Manual, 1 - Auto.
WhiteBalanceArea	Query white balance area. Response: .../WhiteBalanceArea:n . Parameter n (integer) depends on camera controller.
WideDynamicRangeMode	Query wide dynamic range mode. Response: .../WideDynamicRangeMode:n . Parameter n (integer): 0 - Off, 1 - On.
StabilisationMode	Query image stabilization mode inside camera core. Response: .../StabilisationMode:n . Parameter n (integer): 0 - Off, 1 - On.

Command	Description
IsoSensitivity	Query ISO sensitivity. Response: .../IsoSensitivity:n . Parameter n (integer): iso sensitivity Value depends on camera controller.
SceneMode	Query scene mode. Response: .../SceneMode:n . Parameter n (integer): scene mode. Value depends on camera controller.
BrightnessMode	Query brightness mode. Response: .../BrightnessMode:n . Parameter n (integer): 0 - Manual, 1 - Auto.
Brightness	Query brightness level. Response: .../Brightness:n . Parameter n (integer): brightness level [0-100%].
Contrast	Query contrast level. Response: .../Contrast:n . Parameter n (integer): 1 - 100% .
GainMode	Query gain mode. Response: .../GainMode:n . Parameter n (integer): 0 - Manual, 1 - Auto.
Gain	Query gain level. Response: .../Gain:n . Parameter n (integer): gain level [1-100%].
SharpeningMode	Query sharpening mode. Response: .../SharpeningMode:n . Parameter n (integer): 0 - Manual, 1 - Auto.
Sharpening	Query sharpening level. Response: .../Sharpening:n . Parameter n (integer): sharpening level [1-100%].
Palette	Query palette. Response: .../Palette:n . Parameter n (integer) depends on camera controller. Default values: 0 - White hot, 1 - Black hot.
AgcMode	Query analog gain control mode. Response: .../AgcMode:n . Parameter n (integer): 0 - Manual, 1 - Auto.
ShutterMode	Query shutter mode. Response: .../ShutterMode:n . Parameter n (integer): 0 - Manual, 1 - Auto.
ShutterPos	Query shutter position. Response: .../ShutterPosition:n . Parameter n (integer): shutter position 0 (full close) - 65535 (full open).
ShutterSpeed	Query shutter speed. Response: .../ShutterSpeed:n . Parameter n (integer): [0-100%].
ExposureCompensationMode	Query exposure compensation mode. Response: .../ExposureCompensationMode:n . Parameter n (integer): 0 - Off, 1 - On.
ExposureCompensationPos	Query exposure compensation position. Response: .../ExposureCompensationMode:n . Parameter n (integer): exposure compensation position.
DefogMode	Query defog mode. Response: .../DefogMode:n . Parameter n (integer): 0 - Off, 1 - On.

Command	Description
DehazeMode	Query dehaze mode. Response: .../DehazeMode:n . Parameter n (integer): 0 - Off, 1 - On. Particular cameras can have specific values.
NoiseReductionMode	Query noise reduction mode. Response: .../NoiseReductionMode:n . Parameter n : 0 - Off, 1 - 2D, 3 - 3D.
BlackAndWhiteFilterMode	Query black and white filter mode. Response: .../BlackAndWhiteFilterMode:n . Parameter n (integer): 0 - Off, 1 - On.
FilterMode	Query filter mode. Response: .../FilterMode:n . Parameter n depends on camera controller.
NucMode	Query NUC mode for thermal cameras. Response: .../NucMode:n . Parameter n (integer): 0 - Manual, 1 - Auto.
AutoNucIntervalSec	Query auto NUC interval for thermal cameras. Response: .../AutoNucIntervalSec:n . Parameter n (integer): NUC interval [sec] from 0 (Off) to 100000 .
AutoNucIntervalMin	Query auto NUC interval for thermal cameras. Response: .../AutoNucIntervalMin:n . Parameter n (integer): NUC interval [minutes] from 0 (Off) to 100000 .
DdeMode	Query DDE mode. Response: .../DdeMode:n . Parameter n (integer): 0 - Off, 1 - On.
DdeLevel	Query DDE level. Response: .../DdeLevel:n . Parameter n (integer) depends on camera controller.
Temperature	Query camera temperature. Response: .../Temperature:n . Parameter n (integer): temperature [deg] Celsius.
AlcGate	Query ALC gate. Response: .../AlcGate:n . Parameter n (integer) depends on camera controller.
Sensitivity	Query sensor sensitivity. Response: .../Sensitivity:n . Parameter n (integer) depends on camera controller.
ChangingMode	Query changing mode (day / night). Response: .../ChangingMode:n . Parameter n (integer): 0 - Manual, 1 - Auto.
ChangingLevel	Query changing level (day / night). Response: .../ChangingLevel:n . Parameter n (integer) depends on camera controller.
ChromaLevel	Query chroma level. Response: .../ChromaLevel:n . Parameter n (integer): chroma level [0-100%].
Detail	Query details enhancement. Response: .../Detail:n . Parameter n (integer): detail enhancement level [0-100%].
Profile	Query camera settings profile. Response: .../Profile:n . Parameter n (integer) depends on camera controller.

Command	Description
IsConnected	Query connection status. Response: .../IsConnected:n . Parameter n (integer): 0 - no responses from camera, 1 - connected. Shows if we have response from camera.
IsOpen	Query open status. Response: .../IsOpen:n . Parameter n (integer): 1 - open, 0 - not open. Open status shows if the camera controller is initialized or not, but does not indicate whether the camera controller has communication with the camera hardware.
Custom1	Query camera custom parameter. Response: .../Custom1:n (float / integer). Parameter n depends on camera controller.
Custom2	Query camera custom parameter. Response: .../Custom2:n (float / integer). Parameter n depends on camera controller.
Custom3	Query camera custom parameter. Response: .../Custom3:n (float / integer). Parameter n depends on camera controller.
AgcRoiX0	Query AGC ROI left horizontal coordinate. Response: .../AgcRoiX0:n . Parameter n (integer): x0 position, pixels (original camera resolution).
AgcRoiY0	Query AGC ROI top vertical coordinate. Response: .../AgcRoiY0:n . Parameter n (integer): y0 position, pixels (original camera resolution).
AgcRoiX1	Query AGC ROI right horizontal coordinate. Response: .../AgcRoiX1:n . Parameter n (integer): x1 position, pixels (original camera resolution).
AgcRoiY1	Query AGC ROI bottom vertical coordinate. Response: .../AgcRoiY1:n . Parameter n (integer): y1 position, pixels (original camera resolution).
Type	Query camera type. Response: .../Type:string . Parameter string (string): camera type identifier: None, G5, G5Atcom, FlirBoson, FlirTau2, Hitachi, IrayFt, IrayBlock, Kowa, Owl, FujiSx, Sony, Ventus .
CoolerHours	Query thermal camera cooler working hours counter. Response: .../CoolerHours:n . Parameter n (integer): cooler hours counter.

Some example messages:

[client -> system] **ID:17/Command/Camera1/Reset** - Restart Jafar service for camera 1.

[system -> client] **ID:17/Ack** - command accepted.

[client -> system] **ID:18/Command/Camera1/Camera/Profile:2** - Set camera settings profile 2.

[system -> client] **ID:18/Ack** - command accepted.

[client -> system] **ID:19/Command/Camera1/Camera/ChangingMode:1** - Set changing mode (day / night) to auto.

[system -> client] **ID:19/Ack** - command accepted.

[client -> system] **ID:20/Query/Camera2/Camera/ChangingMode** - Query changing mode from camera 2.

[system -> client] **ID:20/Reply/Camera2/Camera/ChangingMode:1** - Changing mode is auto.

[client -> system] **ID:21/Query/Camera3/Camera/IsConnected** - Query connection status.
[system -> client] **ID:21/Reply/Camera3/Camera/IsConnected:1** - Connected.

4 2 3 VideoSource control messages

VideoSource is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules.
VideoSource submodule is responsible for video capture control.

Table 4.2.3.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/VideoSource/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Restart	Restart video capture module.
Custom1:n	Custom video capture parameters 1. Parameter n (float / integer) depends on system.
Custom2:n	Custom video capture parameters 2. Parameter n (float / integer) depends on system.
Custom3:n	Custom video capture parameters 3. Parameter n (float / integer) depends on system.

Table 4.2.3.2 - Query commands. Messages start from **Query/Camera1(2,3,4)/VideoSource/....** Response from system starts with **Reply/Camera1(2,3,4)/VideoSource/....** Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Width	Query original capture video width from camera. Response: .../Width:n . Parameter n (integer): video width, pixels.
Height	Query original capture video height from camera. Response: .../Height:n . Parameter n (integer): video height, pixels.
Fps	Query video capture FPS. Response: .../Fps:n . Parameter n (float): frames per second.
Custom1	Query custom video capture parameters 1. Response: .../Custom1:n . Parameter n (float / integer) depends on system.
Custom2	Query custom video capture parameters 2. Response: .../Custom2:n . Parameter n (float / integer) depends on system.
Custom3	Query custom video capture parameters 3. Response: .../Custom3:n . Parameter n (float / integer) depends on system.

Some example messages:

[client -> system] **ID:22/Command/Camera1/VideoSource/Restart** - Restart video capture module of camera 1.

[system -> client] **ID:22/Ack** - command accepted.

[client -> system] **ID:23/Command/Camera1/VideoSource/Custom1:10** - Set custom parameters of camera 1 capture module.

[system -> client] **ID:23/Ack** - command accepted.

[client -> system] **ID:24/Query/Camera2/VideoSource/Width** - Query video width from camera 2.

[system -> client] **ID:24/Reply/Camera2/VideoSource/Width:1920** - Reply about video width of camera 2.
 [client -> system] **ID:25/Query/Camera2/VideoSource/Height** - Query video height from camera 2.
 [system -> client] **ID:25/Reply/Camera2/VideoSource/Height:1080** - Reply about video height of camera 2.

4 2 4 VideoTracker control messages

VideoTracker is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **VideoTracker** submodule is responsible for video tracking. Each camera has one video tracker channel. It means that each camera can track only one object.

Table 4.2.4.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/VideoTracker/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Capture:XxY	Capture object with scaled relative position. Parameters: X (integer) - horizontal position of tracking rectangle center as permille of stream width (value / 1000 = percentage of stream width), Y (integer) - vertical position of tracking rectangle center as permille of stream height (value / 1000 = percentage of stream height). For example, to capture at the center of the stream, send Capture:50000x50000 . The system automatically adjusts coordinates when the stream aspect ratio differs from the original camera aspect ratio. Note: after capture the tracker will not start pan-tilt unit control. To start pan-tilt unit control use EnablePtControl command.
CapturePercents:XxY	Capture object with relative position. Parameters: X (float) - horizontal position % of tracking rectangle center from video width, Y (float) - vertical position % of tracking rectangle center from video height. This command is used to capture object on video which has resolution different from original camera resolution. Note: after capture the tracker will not start pan-tilt unit control. To start pan-tilt unit control use EnablePtControl command.
CaptureObject:n	Capture object by ID. Parameter n (integer): object ID (usually from 0 to 9). The detectors (motion detector, changes detector or classification) on each camera detect object and assign unique ID which is stable in time. User usually sees detection results on video. To capture particular object for tracking send command with object ID. Example: .../VideoTracker/CaptureObject:5 (capture object with ID 5). The tracker will automatically adjust tracking rectangle size according to object parameters (in valid range [16:128] pixels). Note: before capture you have to set appropriate object detector number for tracker with command .../VideoTracker/ActiveDetector:1 (0 - motion detector, 1 - changes detector, 2 - classification). Note: after capture the tracker will not start pan-tilt unit control. To start pan-tilt unit control use EnablePtControl command.

Command	Description
Reset	Reset tracking. This command will also stop pan-tilt control. Note: the reset command also reset pan-tilt control permission.
EnablePtControl	Enable pan-tilt control after capture.
DisablePtControl	Disable pan-tilt control.
AdjustRectSize	Adjust tracking rectangle size. Video tracker estimates object size and position inside tracking rectangle. The real size of object can be much less than size of tracking rectangle. This command will force video tracker to adjust the size of tracking rectangle once to optimal value if possible.
AdjustRectPos	Adjust tracking rectangle position. Video tracker estimates object size and position inside tracking rectangle. The real position of object inside tracking rectangle may not be optimal. This command will force video tracker to adjust the position of tracking rectangle once if possible.
MoveRect:dXxdY	Move tracking rectangle (change position). Parameters: dX (integer) - add to X coordinate +/-, pixels. 0 - no change. dY (integer) - add to Y coordinate +/-, pixels. 0 - no change.
RectPos:XxY	Set tracking rectangle position. Parameters: X (integer) - Rectangle center horizontal coordinate according to original camera resolution, Y (integer) - Rectangle center vertical coordinate according to original camera resolution. Note: the original camera resolution can be different from what the user receives via RTSP from system. For example, original thermal camera resolution can be 640x512 pixels but resolution of RTSP stream can be 1920x1080 (defined by RTSP server parameters). To get original camera resolution use .../Query/Camera1(2,3,4)/VideoSource/Width and .../Query/Camera1(2,3,4)/VideoSource/Height requests to the system.
RectPosPercents:XxY	Set tracking rectangle position % from video size. Parameters: X (integer) - horizontal position % of tracking rectangle center from video width, Y (integer) - vertical position % of tracking rectangle center from video height. This command is used for video which has resolution different from original camera resolution.
RectSize:WxH	Set tracking rectangle size. Parameters: W (integer) - rectangle width, pixels [16:128] . H (integer) - rectangle height, pixels [16:128] .
ChangeRectSize:dXxdY	Change tracking rectangle size. Parameters: dX (integer) - horizontal size add +/-, pixels. dY (integer) - vertical size add +/-, pixels. The size (width or height) of tracking rectangle will not be less than 16 pixels or more than 128 pixels.

Command	Description
LostModeOption:n	Set option for LOST (object loss mode) mode. Parameter n (integer): parameter that defines the behavior of the tracking algorithm in LOST mode. Default is 0 . Possible values: 0 . In LOST mode, the coordinates of the center of the tracking rectangle are not updated and remain the same as before entering LOST mode. 1 . The coordinates of the center of the tracking rectangle are updated based on the components of the object's speed calculated before going into LOST mode. If the tracking rectangle "touches" the edge of the video frame, the coordinate updating for this component (horizontal or vertical) will stop. 2 . The coordinates of the center of the tracking rectangle are updated based on the components of the object's speed calculated before going into LOST mode. The tracking is reset if the center of the tracking rectangle touches any of the edges of the video frame.
RectAutoSizeMode:n	Set tracking rectangle auto size flag. Parameter n (integer): 0 - no use, 1 - use. Video tracker estimates object size and position inside tracking rectangle. The real size of object can be much less than size of tracking rectangle. If tracking rectangle auto size mode is 1 the tracker will be adjusting tracking rectangle size automatically in TRACKING mode.
RectAutoPosMode:n	Set tracking rectangle auto position flag. Parameters n (integer): 0 - no use, 1 - use. Video tracker estimates object size and position inside tracking rectangle. The real position of object inside tracking rectangle may not be optimal. If tracking rectangle auto position mode is 1 the tracker will be adjusting tracking rectangle position automatically in TRACKING mode.
ActiveCamera:n	Set active camera for pan-tilt control. Parameter n (integer): 0 - Camera 1, 1 - Camera 2, 2 - Camera 3.
MaxPanSpeed:n	Maximum pan speed in automatic control. Parameter n (integer): maximum Pan speed [deg/s].
MaxTiltSpeed:n	Maximum tilt speed in automatic control. Parameter n (integer): maximum Tilt speed [deg/s].
ActiveDetector:n	Active object detector to capture object by ID. Parameter n (integer): 0 - motion detector, 1 - changes detector, 2 - classification. Example: if parameter set to 0 after command .../Camera1/VideoTracker/CaptureObject:4 the tracker of camera 1 will capture object with ID 4 from motion detector.
EnableTracker:n	Enable / disable tracker control. Parameter n (integer): 0 - disable, 1 - enable.

Command	Description
ObjectLossThreshold:n	Criteria for object loss detection. Default value 10% . Parameter n (integer) - probability [0:100%]. The video tracker calculates object detection probability in the range [0:100%]. If current detection probability is less than the threshold, the video tracking algorithm switches to LOST mode.
ObjectDetectionThreshold:n	Criteria for object detection after loss (re-capture object). Default value 50% . Parameter n (integer) - probability [0:100%]. The video tracker calculates object detection probability in the range [0:100%]. If current detection probability is more than threshold and current tracking mode is LOST the tracker will switch to TRACKING mode (re-capture object).
CenteringMode:n	Set tracker centering mode. Parameter n (integer): 0 - disabled (default), 1 - enabled. When enabled, the tracker automatically moves the tracking rectangle toward the center of the frame.

Table 4.2.4.2 - VideoTracker submodule queries. Messages start from **.../Query/Camera1(2,3,4)/VideoTracker/....** Response from system starts with **.../Reply/Camera1(2,3,4)/VideoTracker/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
Mode	Query video tracker mode. Response: .../VideoTracker/Mode:n . Parameter n (integer): 0 - FREE mode, no tracking, 1 - TRACKING mode, 2 - LOST mode, object lost.
RectPos	Query tracking rectangle position. Response: .../VideoTracker/RectPos:XxY . Parameters: X (integer) - Rectangle center horizontal coordinate, pixels on original video width from camera, Y (integer) - Rectangle center vertical coordinate, pixels on original video height from camera Note: the original camera resolution can be different from what the user receives via RTSP from system. For example, original thermal camera resolution can be 640x512 pixels but resolution of RTSP stream can be 1920x1080 (defined by RTSP server parameters). To get original camera resolution use .../Query/Camera1(2,3,4)/VideoSource/Width and .../Query/Camera1(2,3,4)/VideoSource/Height requests to the system.
RectPosPercents	Query tracking rectangle position % from video size. Response: .../VideoTracker/RectPosPercents:XxY . Parameters: X (float) - horizontal position % of tracking rectangle center from video width, Y (float) - vertical position % of tracking rectangle center from video height.
RectSize	Query tracking rectangle size. Response: .../VideoTracker/RectSize:WxH . Parameters: W - rectangle width, pixels [16:128]. H - rectangle height, pixels [16:128].

Command	Description
LostModeOption	Query option for LOST mode. Response: .../VideoTracker/LostModeOption:n . Parameter n (integer): lost mode option which defines the behavior of the tracking algorithm in LOST mode. Default is 0 . Possible values: 0 . In LOST mode, the coordinates of the center of the tracking rectangle are not updated and remain the same as before entering LOST mode. 1 . The coordinates of the center of the tracking rectangle are updated based on the components of the object's speed calculated before going into LOST mode. If the tracking rectangle "touches" the edge of the video frame, the coordinate updating for this component (horizontal or vertical) will stop. 2 . The coordinates of the center of the tracking rectangle are updated based on the components of the object's speed calculated before going into LOST mode. The tracking is reset if the center of the tracking rectangle touches any of the edges of the video frame.
RectAutoSizeMode	Query tracking rectangle auto size flag. Response: .../VideoTracker/RectAutoSizeMode:n . Parameter n (integer): 0 - no use, 1 - use.
RectAutoPosMode	Query tracking rectangle auto position flag. Response: .../VideoTracker/RectAutoPosMode:n . Parameter n (integer): 0 - no use, 1 - use.
ActiveCamera	Query active camera for pan-tilt control. Response: .../VideoTracker/ActiveCamera:n . Parameter n (integer): 0 - Camera 1, 1 - Camera 2, 2 - Camera 3.
MaxPanSpeed	Query maximum Pan speed in automatic control. Parameter n (float): maximum absolute speed [deg/s]. Note : it should not be more than .../MotorControl/Pan/MaxSpeed parameter.
MaxTiltSpeed	Query maximum Tilt speed in automatic control. Parameter n (float): maximum absolute speed [deg/s]. Note : it should not be more than .../MotorControl/Tilt/MaxSpeed parameter.
ActiveDetector	Query active object detector to capture object by ID. Response: .../VideoTracker/ActiveDetector:n . Parameter n : 0 - motion detector, 1 - changes detector, 2 - classification.
EnableTracker	Query enable status of tracker control. Response: .../VideoTracker/EnableTracker:n . Parameter n (integer): 0 - disable, 1 - enable.
ObjectLossThreshold	Query criteria for object loss detection. Response: .../VideoTracker/ObjectLossThreshold:n . Parameter n (integer) - probability [0:100%]. The video tracker calculates object detection probability in the range [0:100%]. If current detection probability is less than the threshold, the video tracking algorithm switches to LOST mode.

Command	Description
ObjectDetectionThreshold	Query criteria for object detection after loss (re-capture object). Response: .../VideoTracker/ObjectDetectionThreshold:n . Parameter n (integer) - probability [0:100%]. The video tracker calculates object detection probability in the range [0:100%]. If current detection probability is more than threshold and current tracking mode is LOST the tracker will switch to TRACKING mode (re-capture object).
CenteringMode	Query tracker centering mode. Response: .../VideoTracker/CenteringMode:n . Parameter n (integer): 0 - disabled, 1 - enabled.

Some example messages:

[client -> system] **ID:26/Command/Camera1/VideoTracker/Reset** - Reset video tracking on camera 1.

[system -> client] **ID:26/Ack** - command accepted.

[client -> system] **ID:27/Command/Camera1/VideoTracker/CapturePercents:20.4x36.7** - Capture object on camera 1 with position 20.4% from video width and 36.7% from video height.

[system -> client] **ID:27/Ack** - command accepted.

[client -> system] **ID:28/Query/Camera2/VideoTracker/Mode** - Query video tracker mode from camera 2.

[system -> client] **ID:28/Reply/Camera2/VideoTracker/Mode:0** - FREE video tracker mode of camera 2.

[client -> system] **ID:29/Query/Camera2/VideoTracker/LostModeOption** - Query video tracker lost mode option from camera 2.

[system -> client] **ID:29/Reply/Camera2/VideoTracker/LostModeOption:0** - Lost mode option 0 of camera 2.

4 2 5 VideoStabiliser control messages

VideoStabiliser is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules.

VideoStabiliser submodule is responsible for digital video stabilization. Each camera applies stabilization after image flip and digital zoom.

Table 4.2.5.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/VideoStabiliser/...** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Reset	Reset stabilization algorithm.
HoldSec:n	Hold stabilization algorithm for certain time. Parameter n (integer): hold time [sec]. After hold command the stabilization algorithm will wait n second to start processing again. This functions works automatically in when pan-tilt moves (hold for 2 seconds).
On	Enable video stabilization.
Off	Disable video stabilization.

Command	Description
XOffsetLimit:n	Set maximum horizontal offset, pixels. Parameter n (integer): maximum horizontal offset, [pixels], which will be compensated. If detected offset bigger than parameter value the camera will compensate only XOffsetLimit pixels.
YOffsetLimit:n	Set maximum vertical offset, pixels. Parameter n (integer): maximum vertical offset, [pixels], which will be compensated. If detected offset bigger than parameter value the camera will compensate only YOffsetLimit pixels.
AOffsetLimit:n	Set maximum rotation angle, degrees. Parameter n (float): maximum rotation angle, [deg], which will be compensated.
Mode:n	Set video stabilizer mode. Parameter n (integer): 0 - disabled, 1 - enabled.
TransparentBorderMode:n	Set transparent borders mode. Parameter n (integer): 0 - disabled, 1 - enabled.
ConstXOffset:n	Set constant horizontal image offset in pixels. Parameter n (integer): horizontal offset, pixels. It used for boresight correction.
ConstYOffset:n	Set constant vertical image offset in pixels. Parameter n (integer): vertical offset, pixels. It used for boresight correction.
ConstAOffset:n	Set constant rotational angle. Parameter n (float): rotation angle, [deg]. It used for boresight correction.
Type:n	Set algorithm type. Parameter n (integer): 0 - 2D type 1. Good for low contrast images. Stabilization only on horizontal and vertical. 1 - 2D type 2. Stabilization only on horizontal and vertical. 2 - 3D. Stabilization on horizontal and vertical + rotation.

Table 4.2.5.2 - Query commands. Messages start from **Query/Camera1(2,3,4)/VideoStabiliser/...** Response from system starts with **Reply/Camera1(2,3,4)/VideoStabiliser/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
XOffsetLimit	Query maximum horizontal offset, pixels. Response: .../VideoStabiliser/XOffsetLimit:n . Parameter n (integer): maximum horizontal offset, pixels, which will be compensated.
YOffsetLimit	Query maximum vertical offset, pixels. Response: .../VideoStabiliser/YOffsetLimit:n . Parameter n (integer): maximum vertical offset, pixels, which will be compensated.
AOffsetLimit	Query maximum rotation angle, degrees. Response: .../VideoStabiliser/AOffsetLimit:n . Parameter n (float): maximum rotation angle, degrees, which will be compensated.

Command	Description
Mode	Query video stabilizer mode. Response: .../VideoStabiliser/Mode:n . Parameter n (integer): 0 - disabled, 1 - enabled.
TransparentBorderMode	Query transparent borders mode. Response: .../VideoStabiliser/TransparentBorderMode:n . Parameter n (integer): 0 - disabled, 1 - enabled.
ConstXOffset	Query constant horizontal image offset in pixels. Response: .../VideoStabiliser/ConstXOffset:n . Parameter n (integer): horizontal offset, pixels. It used for boresight correction.
ConstYOffset	Query constant vertical image offset in pixels. Response: .../VideoStabiliser/ConstYOffset:n . Parameter n (integer): vertical offset, pixels. It used for boresight correction.
ConstAOffset	Query constant rotational angle. Response: .../VideoStabiliser/ConstAOffset:n . Parameter n (float): rotation angle, [deg]. It used for boresight correction.
Type	Query algorithm type. Response: .../VideoStabiliser/Type:n . Parameter n (integer): 0 - 2D type 1. Good for low contrast images. Stabilization only on horizontal and vertical. 1 - 2D type 2. Stabilization only on horizontal and vertical. 2 - 3D. Stabilization on horizontal and vertical + rotation.

Some example messages:

[client -> system] **ID:26/Command/Camera1/VideoStabiliser/Reset** - Reset video stabilizer of camera 1.

[system -> client] **ID:26/Ack** - command accepted.

[client -> system] **ID:27/Command/Camera1/VideoStabiliser/TransparentBorderMode:1** - Set transparent border for stabilization of camera 1.

[system -> client] **ID:27/Ack** - command accepted.

[client -> system] **ID:28/Query/Camera2/VideoStabiliser/Mode** - Query video stabilizer mode from camera 2.

[system -> client] **ID:28/Reply/Camera2/VideoStabiliser/Mode:1** - Reply about stabilizer mode of camera 2.

[client -> system] **ID:29/Query/Camera2/VideoStabiliser/ConstXOffset** - Query constant horizontal offset from camera 2.

[system -> client] **ID:29/Reply/Camera2/VideoStabiliser/ConstXOffset:0** - Reply about constant horizontal offset from camera 2.

4 2 6 MotionDetector control messages

MotionDetector is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **MotionDetector** submodule is responsible for detecting of moving objects.

Table 4.2.6.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/MotionDetector/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Reset	Reset motion detector. System resets detector automatically when pan-tilt moves.
On	Enable motion detector.
Off	Disable motion detector.
FrameBufferSize:n	Set frame buffer size. Parameter n (integer): frame buffer size. Frame buffer size is used to remove noise of small movement from video before detection. Default value 5 . The larger the frame buffer size, the more robust the motion detector will be in case noise. For example, for water surface it is recommended to set the value to 10 .
MinObjectWidth:n	Set minimum object width, pixels. Parameter n (integer): minimum object width of object [pixels] to be detected.
MaxObjectWidth:n	Set maximum object width, pixels. Parameter n (integer): maximum object width of object [pixels] to be detected.
MinObjectHeight:n	Set minimum object height, pixels. Parameter n (integer): minimum object height of object [pixels] to be detected.
MaxObjectHeight:n	Set maximum object height, pixels. Parameter n (integer): maximum object height of object [pixels] to be detected.
XDetectionCriteria:n	Set horizontal detection criteria. Parameter n (integer): the number of video frames at which the object must move in the horizontal (velocity component) direction to be detected. This parameter also is used to exclude from detection constantly moving objects like trees.
YDetectionCriteria:n	Set vertical detection criteria. Parameter n (integer): the number of video frames at which the object must move in the vertical (velocity component) direction to be detected. This parameter also is used to exclude from detection constantly moving objects like trees.
ResetCriteria:n	Set reset criteria. Parameter n (integer): the number of video frames during which the object must not be detected for the system to remove it from the detection results. The recommended value is 5 . The object will not be removed from the detection results if it is missing from the video for a period of time shorter than the parameter value.
Sensitivity:n	Set sensitivity level. Parameter n (integer): pixel value deviation [0-255]. Recommended value is 10 for daylight cameras and 15 for thermal cameras. The motion detector decides if there is motion in each pixel. The lower the value, the higher the sensitivity but also the higher the probability of false detections.
MinXSpeed:n	Set minimum horizontal speed for object filtering. Parameter n (float): minimum horizontal speed. Objects with horizontal speed less than this value will be excluded from detection results.
MaxXSpeed:n	Set maximum horizontal speed for object filtering. Parameter n (float): maximum horizontal speed. Objects with horizontal speed greater than this value will be excluded from detection results.
MinYSpeed:n	Set minimum vertical speed for object filtering. Parameter n (float): minimum vertical speed. Objects with vertical speed less than this value will be excluded from detection results.
MaxYSpeed:n	Set maximum vertical speed for object filtering. Parameter n (float): maximum vertical speed. Objects with vertical speed greater than this value will be excluded from detection results.
Mode:n	Set motion detector mode. Parameter n (integer): 0 - disabled, 1 - enabled.
Objects/Subscribe	Note: only for UDP control. Subscribe for detection results. The user can subscribe to the detection results (list of objects). To do this, the user should send a command from any UDP port. After receiving the command, the system will remember the user's IP and UDP port number. After that, if the MotionDetector is active the system will send the list of objects for each frame of video. Message format: ID:1(2,3)/Notify/Camera1(2,3,4)/MotionDetector/Objects:N(Id;Type;X;Y;Width;Height) (Id;Type;X;Y;Width;Height). Parameters: ID:1 - Camera1, ID:2 - Camera2, ID:3 - Camera3. N - number of detected objects. If not detection the value will be 0. If there are detection after will be block with objects information (maximum 10 objects, the same as displayed on video): ID - unique object ID the same as displayed on video. Type - always 0. X - top-left horizontal coordinate (pixels) of object's bounding box (according to original camera resolution). Y - top-left vertical coordinate (pixels) of object's bounding box (according to original camera resolution) (window coordinate system, top->bottom, left->right). Width - object's bounding box width (according to original camera resolution). Height - object's bounding box height (according to original camera resolution).
Objects/Unsubscribe	Unsubscribe from detection results.

Table 4.2.6.2 - MotionDetector submodule queries. Messages start from **Query/Camera1(2,3,4)/MotionDetector/....** Response from system starts with **Reply/Camera1(2,3,4)/MotionDetector/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
FrameBufferSize	Query frame buffer size. Response: .../MotionDetector/FrameBufferSize:n. Parameter n (integer): frame buffer size.
MinObjectWidth	Query minimum object width, pixels. Response: .../MotionDetector/MinObjectWidth:n. Parameter n (integer): minimum object width of object [pixels] to be detected.
MaxObjectWidth	Query maximum object width, pixels. Response: .../MotionDetector/MaxObjectWidth:n. Parameter n (integer): maximum object width of object [pixels] to be detected.
MinObjectHeight	Query minimum object height, pixels. Response: .../MotionDetector/MinObjectHeight:n. Parameter n (integer): minimum object height of object [pixels] to be detected.
MaxObjectHeight	Query maximum object height, pixels. Response: .../MotionDetector/MaxObjectHeight:n. Parameter n (integer): maximum object height of object [pixels] to be detected.
XDetectionCriteria	Query horizontal detection criteria. Response: .../MotionDetector/XDetectionCriteria:n. Parameter n (integer): the number of video frames at which the object must move in the horizontal (velocity component) direction to be detected.
YDetectionCriteria	Query vertical detection criteria. Response: .../MotionDetector/YDetectionCriteria:n. Parameter n (integer): the number of video frames at which the object must move in the vertical (velocity component) direction to be detected.
ResetCriteria	Query reset criteria. Response: .../MotionDetector/ResetCriteria:n. Parameter n (integer): the number of video frames during which the object must not be detected for the system to remove it from the detection results.
Sensitivity	Query sensitivity level. Response: .../MotionDetector/Sensitivity:n. Parameter n (integer): pixel value deviation from 0 to 255.
Mode	Query motion detector mode. Response: .../MotionDetector/Mode:n. Parameter n (integer): 0 - disabled, 1 - enabled.

Command	Description
Objects	Note: only for UDP control. Query detected objects. Response: .../MotionDetector/Objects:N(Id;Type;X;Y;Width;Height) (Id;Type;X;Y;Width;Height). Parameters: N - number of detected objects. If not detection the value will be 0. If there are detection after will be block with objects information (maximum 10 objects, the same as displayed on video): ID - unique object ID the same as displayed on video. Type - always 0. X - top-left horizontal coordinate (pixels) of object's bounding box (according to original camera resolution). Y - top-left vertical coordinate (pixels) of object's bounding box (according to original camera resolution) (window coordinate system, top->bottom, left->right). Width - object's bounding box width (according to original camera resolution). Height - object's bounding box height (according to original camera resolution).

Some example messages:

[client -> system] **ID:30/Command/Camera1/MotionDetector/Reset** - Reset motion detector of camera 1.

[system -> client] **ID:30/Ack** - Command accepted.

[client -> system] **ID:31/Command/Camera1/MotionDetector/MaxObjectWidth:100** - Set maximum object width 100 pixels for motion detection for camera 1.

[system -> client] **ID:31/Ack** - Command accepted.

[client -> system] **ID:32/Query/Camera2/MotionDetector/Mode** - Query motion detector mode from camera 2.

[system -> client] **ID:32/Reply/Camera2/MotionDetector/Mode:1** - Reply about motion detector mode of camera 2.

[client -> system] **ID:33/Query/Camera2/MotionDetector/ResetCriteria** - Query reset criteria of motion detector from camera 2.

[system -> client] **ID:33/Reply/Camera2/MotionDetector/ResetCriteria:5** - Reply about reset criteria from camera 2.

4 2 7 ChangesDetector control messages

ChangesDetector is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **ChangesDetector** submodule is responsible for detecting changes on video (detection not moving objects).

Table 4.2.7.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/ChangesDetector/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Reset	Reset changes detector. System resets detector automatically when pan-tilt moves.
On	Enable changes detector.
Off	Disable changes detector.

Command	Description
FrameBufferSize:n	Set frame buffer size. Parameter n (integer): frame buffer size. Frame buffer size is used to remove noise of small movement from video before detection. Default value 5 . The larger the frame buffer size, the more robust the changes detector will be to noise. For example, for water surface it is recommended to set the value to 10 .
MinObjectWidth:n	Set minimum object width, pixels. Parameter n (integer): minimum object width of object [pixels] to be detected.
MaxObjectWidth:n	Set maximum object width, pixels. Parameter n (integer): maximum object width of object [pixels] to be detected.
MinObjectHeight:n	Set minimum object height, pixels. Parameter n (integer): minimum object height of object [pixels] to be detected.
MaxObjectHeight:n	Set maximum object height, pixels. Parameter n (integer): maximum object height of object [pixels] to be detected.
XDetectionCriteria:n	Set detection criteria. XDetectionCriteria and YDetectionCriteria are identical, user can use any of them. Parameter n (integer): the number of video frames at which the object must appears in detection to be detected (added to detection list).
YDetectionCriteria:n	Set detection criteria. XDetectionCriteria and YDetectionCriteria are identical, user can use any of them. Parameter n (integer): the number of video frames at which the object must appears in detection to be detected (added to detection list).
ResetCriteria:n	Set reset criteria. Parameter n (integer): the number of video frames during which the object must not be detected for the system to remove it from the detection results. The recommended value is 5 . The object will not be removed from the detection results if it is missing from the video for a period of time shorter than the parameter value.
Sensitivity:n	Set sensitivity level. Parameter n (integer): pixel value deviation from [0-255]. Recommended value is 10 for daylight cameras and 15 for thermal cameras. The changes detector decides if there is change in each pixel. The lower the value, the higher the sensitivity but also the higher the probability of false detections.
Mode:n	Set changes detector mode. Parameter n (integer): 0 - disabled, 1 - enabled.
Objects/Subscribe	Note: only for UDP control. Subscribe for detection results. The user can subscribe to the detection results (list of objects). To do this, the user should send a command from any UDP port. After receiving the command, the system will remember the user's IP and UDP port number. After that, if the ChangesDetector is active the system will send the list of objects for each frame of video. Message format: ID:1(2,3)/Notify/Camera1(2,3,4)/ChangesDetector/Objects:N(Id;Type;X;Y;Width;Height) (Id;Type;X;Y;Width;Height). Parameters: ID:1 - Camera1, ID:2 - Camera2, ID:3 - Camera3. N - number of detected objects. If not detection the value will be 0. If there are detection after will be block with objects information (maximum 10 objects, the same as displayed on video): ID - unique object ID the same as displayed on video. Type - always 0. X - top-left horizontal coordinate (pixels) of object's bounding box (according to original camera resolution). Y - top-left vertical coordinate (pixels) of object's bounding box (according to original camera resolution) (window coordinate system, top->bottom, left->right). Width - object's bounding box width (according to original camera resolution). Height - object's bounding box height (according to original camera resolution).
Objects/Unsubscribe	Unsubscribe from detection results.

Table 4.2.7.2 - ChangesDetector submodule queries. Messages start from **Query/Camera1(2,3,4)/ChangesDetector/....** Response from system starts with **Reply/Camera1(2,3,4)/ChangesDetector/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
FrameBufferSize	Query frame buffer size. Response: .../ChangesDetector/FrameBufferSize:n. Parameter n (integer): frame buffer size.
MinObjectWidth	Query minimum object width, pixels. Response: .../ChangesDetector/MinObjectWidth:n. Parameter n (integer): minimum object width of object (pixels) to be detected.

Command	Description
MaxObjectWidth	Query maximum object width, pixels. Response: .../ChangesDetector/MaxObjectWidth:n . Parameter n (integer): maximum object width of object (pixels) to be detected.
MinObjectHeight	Query minimum object height, pixels. Response: .../ChangesDetector/MinObjectHeight:n . Parameter n (integer): minimum object height of object (pixels) to be detected.
MaxObjectHeight	Query maximum object height, pixels. Response: .../ChangesDetector/MaxObjectHeight:n . Parameter n (integer): maximum object height of object (pixels) to be detected.
XDetectionCriteria	Query detection criteria. XDetectionCriteria and YDetectionCriteria are identical, user can use any of them. Response: .../ChangesDetector/XDetectionCriteria:n . Parameter n (integer): the number of video frames at which the object must be detected to be represented in output objects list.
YDetectionCriteria	Query detection criteria. XDetectionCriteria and YDetectionCriteria are identical, user can use any of them. Response: .../ChangesDetector/YDetectionCriteria:n . Parameter n (integer): the number of video frames at which the object must be detected to be represented in output objects list.
ResetCriteria	Query reset criteria. Response: .../ChangesDetector/ResetCriteria:n . Parameter n (integer): the number of video frames during which the object must not be detected for the system to remove it from the detection results.
Sensitivity	Query sensitivity level. Response: .../ChangesDetector/Sensitivity:n . Parameter n (integer): pixel value deviation from 0 to 255.
Mode	Query changes detector mode. Response: .../ChangesDetector/Mode:n . Parameter n (integer): 0 - disabled, 1 - enabled.
Objects	Note: only for UDP control. Query detected objects. Response: ".../ChangesDetector/Objects:N(Id;Type;X;Y;Width;Height)(Id;Type;X;Y;Width;Height)". Parameters: N - number of detected objects. If not detection the value will be 0. If there are detection after will be block with objects information (maximum 10 objects, the same as displayed on video): ID - unique object ID the same as displayed on video. Type - always 0. X - top-left horizontal coordinate (pixels) of object's bounding box (according to original camera resolution). Y - top-left vertical coordinate (pixels) of object's bounding box (according to original camera resolution) (window coordinate system, top->bottom, left->right). Width - object's bounding box width (according to original camera resolution). Height - object's bounding box height (according to original camera resolution).

Some example messages:

[client -> system] **ID:34/Command/Camera1/ChangesDetector/Reset** - Reset changes detector of camera 1.

[system -> client] **ID:34/Ack** - Command accepted.

[client -> system] **ID:35/Command/Camera1/ChangesDetector/MaxObjectWidth:100** - Set maximum object width 100 for changes detection for camera 1.

[system -> client] **ID:35/Ack** - Command accepted.

[client -> system] **ID:36/Query/Camera2/ChangesDetector/Mode** - Query changes detector mode from camera 2.

[system -> client] **ID:36/Reply/Camera2/ChangesDetector/Mode:1** - Reply about changes detector mode of camera 2.

[client -> system] **ID:37/Query/Camera2/ChangesDetector/ResetCriteria** - Query reset criteria of changes detector from camera 2.

[system -> client] **ID:37/Reply/Camera2/ChangesDetector/ResetCriteria:5** - Reply about reset criteria from camera 2.

4 2 8 Classification control messages

Classification is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules.

Classification submodule is responsible for detecting different classes of objects.

Table 4.2.8.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/Classification/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Reset	Reset classification.
On	Enable classification.
Off	Disable classification.
MinObjectWidth:n	Set minimum object width, pixels. Parameter n (integer): minimum object width of object (pixels) to be detected. Minimum value 1 .
MaxObjectWidth:n	Set maximum object width, pixels. Parameter n (integer): maximum object width of object (pixels) to be detected. Minimum value 1 .
MinObjectHeight:n	Set minimum object height, pixels. Parameter n (integer): minimum object height of object (pixels) to be detected. Minimum value 1 .
MaxObjectHeight:n	Set maximum object height, pixels. Parameter n (integer): maximum object height of object (pixels) to be detected. Minimum value 1 .
MinDetectionProbability:n	Set threshold for object detection probability. Parameter n (integer): probability threshold [0-100%]. Only objects with probability greater than MinDetectionProbability will be detected.
Mode:n	Set classification mode. Parameter n : 0 - disabled, 1 - enabled.
XDetectionCriteria:n	Set detection criteria. XDetectionCriteria and YDetectionCriteria are identical, user can use any of them. Parameter n (integer): the number of video frames at which the object must appears in detection to be detected (added to detection list). Minimum value 1 .
YDetectionCriteria:n	Set detection criteria. XDetectionCriteria and YDetectionCriteria are identical, user can use any of them. Parameter n (integer): the number of video frames at which the object must appears in detection to be detected (added to detection list). Minimum value 1 .
ResetCriteria:n	Set reset criteria. Parameter n (integer): the number of video frames during which the object must not be detected for the system to remove it from the detection results. The recommended value is 5 . The object will not be removed from the detection results if it is missing from the video for a period of time shorter than the parameter value. Minimum value 1 .

Command	Description
Objects/Subscribe	Note: only for UDP control. Subscribe for detection results. The user can subscribe to the detection results (list of objects). To do this, the user should send a command from any UDP port. After receiving the command, the system will remember the user's IP and UDP port number. After that, if the Classification is active the system will send the list of objects for each frame of video. Message format: ID:1(2,3)/Notify/Camera1(2,3,4)/Classification/Objects:N(Id;Type;X;Y;Width;Height) (Id;Type;X;Y;Width;Height). Parameters: N - number of detected objects. If not detection the value will be 0. If there are detection after will be block with objects information (maximum 10 objects, the same as displayed on video): ID - unique object ID the same as displayed on video. Type - string object type (e.g. CAR, PERSON etc.). X - top-left horizontal coordinate (pixels) of object's bounding box (according to original camera resolution). Y - top-left vertical coordinate (pixels) of object's bounding box (according to original camera resolution) (window coordinate system, top->bottom, left->right). Width - object's bounding box width (according to original camera resolution). Height - object's bounding box height (according to original camera resolution).
Objects/Unsubscribe	Unsubscribe from detection results.

Table 4.2.8.2 - Classification submodule queries. Messages start from **Query/Camera1(2,3,4)/Classification/....** Response from system starts with **Reply/Camera1(2,3,4)/Classification/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
MinObjectWidth	Query minimum object width, pixels. Response: .../Classification/MinObjectWidth:n. Parameter n - minimum object width of object (pixels) to be detected.
MaxObjectWidth	Query maximum object width, pixels. Response: .../Classification/MaxObjectWidth:n. Parameter n - maximum object width of object (pixels) to be detected.
MinObjectHeight	Query minimum object height, pixels. Response: .../Classification/MinObjectHeight:n. Parameter n - minimum object height of object (pixels) to be detected.
MaxObjectHeight	Query maximum object height, pixels. Response: .../Classification/MaxObjectHeight:n. Parameter n - maximum object height of object (pixels) to be detected.
MinDetectionProbability	Query threshold for object detection probability. Response: .../Classification/MinDetectionProbability:n. Parameter n (integer): probability threshold [0-100%].
Mode	Query classification mode. Response: .../Classification/Mode:n. Parameter n : 0 - disabled, 1 - enabled.
XDetectionCriteria	Query detection criteria. XDetectionCriteria and YDetectionCriteria are identical, user can use any of them. Response: .../Classification/XDetectionCriteria:n. Parameter n (integer): the number of video frames at which the object must be detected to be represented in output objects list.

Command	Description
YDetectionCriteria	Query detection criteria. XDetectionCriteria and YDetectionCriteria are identical, user can use any of them. Response: .../Classification/YDetectionCriteria:n . Parameter n (integer): the number of video frames at which the object must be detected to be represented in output objects list.
ResetCriteria	Query reset criteria. Response: .../Classification/ResetCriteria:n . Parameter n (integer): the number of video frames during which the object must not be detected for the system to remove it from the detection results.
Objects	Note: only for UDP control. Query detected objects. Response: ".../Classification/Objects:N(Id;Type;X;Y;Width;Height)(Id;Type;X;Y;Width;Height)". Parameters: N - number of detected objects. If not detection the value will be 0. If there are detection after will be block with objects information (maximum 10 objects, the same as displayed on video): ID - unique object ID the same as displayed on video. Type - string object type (e.g. CAR, PERSON etc.). X - top-left horizontal coordinate (pixels) of object's bounding box (according to original camera resolution). Y - top-left vertical coordinate (pixels) of object's bounding box (according to original camera resolution) (window coordinate system, top->bottom, left->right). Width - object's bounding box width (according to original camera resolution). Height - object's bounding box height (according to original camera resolution).

Some example messages:

[client -> system] **ID:38/Command/Camera1/Classification/Reset** - Reset classification of camera 1.

[system -> client] **ID:38/Ack** - Command accepted.

[client -> system] **ID:39/Command/Camera1/Classification/MaxObjectWidth:100** - Set maximum object width 100 for classification for camera 1.

[system -> client] **ID:39/Ack** - Command accepted.

[client -> system] **ID:40/Query/Camera2/Classification/Mode** - Query classification mode from camera 2.

[system -> client] **ID:40/Reply/Camera2/Classification/Mode:1** - Reply about classification mode of camera 2.

[client -> system] **ID:41/Query/Camera2/Classification/MinDetectionProbability** - Query detection probability threshold of classification from camera 2.

[system -> client] **ID:41/Reply/Camera2/Classification/MinDetectionProbability:50** - Reply about detection probability threshold from camera 2.

4 2 9 DigitalZoom control messages

DigitalZoom is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **DigitalZoom** submodule is responsible for digital zoom.

Table 4.2.9.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/DigitalZoom/...** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**. For some internal and combined-command scenarios the same parameters can also be addressed via **.../Command/Camera1(2,3,4)/Vpp/DigitalZoom/...**

Command	Description
On	Enable digital zoom.
Off	Disable digital zoom.
Level:n	Set digital zoom level. Parameter n (integer): digital zoom level x10 [10-200], 0.1 magnification level accuracy.
LevelAdd:n	Set digital zoom level increment/decrement. Parameter n (integer): signed digital zoom level increment x10, 0.1 magnification level accuracy.
Mode:n	Set digital zoom mode. Parameter n (integer): 0 - disabled, 1 - enabled.

Table 4.2.9.2 - **DigitalZoom** submodule queries. Messages start from **Query/Camera1(2,3,4)/DigitalZoom/...** Response from system starts with **Reply/Camera1(2,3,4)/DigitalZoom/...** Expected response time: **IP ping time to the system + 5 msec**. For some internal queries, the equivalent address **.../Query/Camera1(2,3,4)/Vpp/DigitalZoom/...** is also accepted.

Command	Description
Level	Query digital zoom level. Response: .../DigitalZoom/Level:n . Parameter n (integer): digital zoom level x10 [10-200], 0.1 magnification level accuracy.
Mode	Query digital zoom mode. Response: .../DigitalZoom/Mode:n Parameter n (integer): 0 - disabled, 1 - enabled.

Some example messages:

[client -> system] **ID:42/Command/Camera1/DigitalZoom/On** - Enable digital zoom in camera 1.

[system -> client] **ID:42/Ack** - Command accepted.

[client -> system] **ID:43/Command/Camera1/DigitalZoom/Level:1.5** - Set digital zoom level for camera 1.

[system -> client] **ID:43/Ack** - Command accepted.

[client -> system] **ID:44/Query/Camera2/DigitalZoom/Mode** - Query digital zoom mode from camera 2.

[system -> client] **ID:44/Reply/Camera2/DigitalZoom/Mode:1** - Reply about digital zoom mode of camera 2.

[client -> system] **ID:45/Query/Camera2/DigitalZoom/Level** - Query digital zoom level from camera 2.

[system -> client] **ID:45/Reply/Camera2/DigitalZoom/Level:1.5** - Reply about digital zoom level from camera 2.

4 2 10 ImageFlip control messages

ImageFlip is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **ImageFlip** submodule is responsible for image flipping.

Table 4.2.10.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/ImageFlip/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Mode:n	Set image flip mode. Parameter n (integer): 0 - no flip, 1 - left-right, 2 - top-bottom, 3 - top-bottom and left-right.

Table 4.2.10.2 - **ImageFlip** submodule queries. Messages start from **Query/Camera1(2,3,4)/ImageFlip/....** Response from system starts with **Reply/Camera1(2,3,4)/ImageFlip/....** Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Mode	Query image flip mode. Response: .../ImageFlip/Mode:n . Parameter n (integer): 0 - no flip, 1 - left-right, 2 - top-bottom, 3 - top-bottom and left-right.

Some example messages:

[client -> system] **ID:46/Command/Camera1/ImageFlip/Mode:1** - Set left-right image flip in camera 1.

[system -> client] **ID:46/Ack** - Command accepted.

[client -> system] **ID:47/Query/Camera2/ImageFlip/Mode** - Query image flip mode from camera 2.

[system -> client] **ID:47/Reply/Camera2/ImageFlip/Mode:1** - Reply about image flip mode of camera 2.

4 2 11 ColorFilter control messages

ColorFilter is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **ColorFilter** submodule is responsible for color filtering.

Table 4.2.11.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/ColorFilter/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
On	Enable color filter.
Off	Disable color filter.
Reset	Reset color filter params. All parameters will be switched to default values: Palette - original, white balance - 50% (no processing), brightness - 50% (no processing), contrast - 50% (no processing), saturation - 50% (no processing).
Mode:n	Set color filter mode. Parameter n (integer): 0 - disabled, 1 - enabled. If the filter is disabled the filter will skip any processing.

Command	Description
Palette:n	Set palette type. Parameter n (integer): 0 - [DEFAULT] original video (without processing), 1 - Inverse, 2 - Ironbow, 3 - Rainbow, 4 - Arctic.
WhiteBalance:n	Set white balance. Parameter n (integer): white balance level [0-100%]. 50% - no white balance processing (original video).
WhiteBalanceAdd:n	Add white balance value. Parameter n (integer): signed white balance level add [-100-100%].
Contrast:n	Set contrast. Parameter n (integer): contrast value [0-100%]. 50% - no contrast processing (original video).
ContrastAdd:n	Add contrast value. Parameter n (integer): signed contrast level add [-100-100%].
Brightness:n	Set brightness. Parameter n (integer): brightness value [0-100%]. 50% - no brightness processing (original video).
BrightnessAdd:n	Add brightness value. Parameter n (integer): signed brightness level add [-100-100%].
Saturation:n	Set saturation. Parameter n (integer): saturation value [0-100%]. 50% - no saturation processing (original video).
SaturationAdd:n	Add saturation value. Parameter n (integer): signed saturation level add [-100-100%].

Table 4.2.11.2 - ColorFilter submodule queries. Messages start from **Query/Camera1(2,3,4)/ColorFilter/....** Response from system starts with **Reply/Camera1(2,3,4)/ColorFilter/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
Mode	Query color filter mode. Response: .../ColorFilter/Mode:n . Parameter n (integer): 0 - disabled, 1 - enabled.
Palette	Query palette type. Response: .../ColorFilter/Palette:n . Parameter n (integer): 0 - [DEFAULT] original video (without processing), 1 - Inverse, 2 - Ironbow, 3 - Rainbow, 4 - Arctic.
WhiteBalance	Query white balance level. Response: .../ColorFilter/WhiteBalance:n . Parameter n (integer): white balance level [0-100%]. 50% - no white balance processing (original video).
Contrast	Query contrast value. Response: .../ColorFilter/Contrast:n . Parameter n (integer): contrast value [0-100%]. 50% - no contrast processing (original video).
Brightness	Query brightness value. Response: .../ColorFilter/Brightness:n . Parameter n (integer): brightness value [0-100%]. 50% - no brightness processing (original video).
Saturation	Query saturation value. Response: .../ColorFilter/Saturation:n . Parameter n (integer): saturation value [0-100%]. 50% - no saturation processing (original video).

Some example messages:

[client -> system] **ID:48/Command/Camera1/ColorFilter/On** - Enable color filter in camera 1.

[system -> client] **ID:48/Ack** - Command accepted.

[client -> system] **ID:49/Command/Camera1/ColorFilter/Palette:1** - Set Inverse palette type in camera 1.

[system -> client] **ID:49/Ack** - Command accepted.

[client -> system] **ID:50/Query/Camera2/ColorFilter/Mode** - Query palette mode from camera 2.

[system -> client] **ID:50/Reply/Camera2/ColorFilter/Mode:1** - Reply about palette mode of camera 2.

[client -> system] **ID:51/Query/Camera2/ColorFilter/Palette** - Query palette type from camera 2.

[system -> client] **ID:51/Reply/Camera2/ColorFilter/Palette:1** - Reply about palette type from camera 2.

4 2 12 Clahe control messages

Clahe is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **Clahe** submodule is responsible for image enhancement (defog / dehaze).

Table 4.2.12.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/Clahe/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
On	Enable clahe.
Off	Disable clahe.
Mode:n	Set clahe mode. Parameter n (integer): 0 - disabled, 1 - enabled.
Type:n	Set clahe type. Parameter n (integer): 0 - global histogram algorithm, 1 - window histogram algorithm.
Level:n	Set clahe level. Parameter n (integer): clahe level [1-100%].
LevelAdd:n	Add clahe level. Parameter n (integer): clahe level add [-100-100%].

Table 4.2.12.2 - **Clahe** submodule queries. Messages start from **Query/Camera1(2,3,4)/Clahe/....** Response from system starts with **Reply/Camera1(2,3,4)/Clahe/....** Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Mode	Query clahe mode. Response: .../Clahe/Mode:n . Parameter n (integer): 0 - disabled, 1 - enabled.
Type	Query clahe type. Response: .../Clahe/Type:n . Parameter n (integer): 0 - global histogram algorithm, 1 - window histogram algorithm.
Level	Query clahe level. Response: .../Clahe/Level:n . Parameter n (integer): clahe level [1-100%].

Some example messages:

[client -> system] **ID:52/Command/Camera1/Clahe/On** - Enable clahe filter in camera 1.
[system -> client] **ID:52/Ack** - Command accepted.
[client -> system] **ID:53/Command/Camera1/Clahe/Type:1** - Set clahe type in camera 1.
[system -> client] **ID:53/Ack** - Command accepted.
[client -> system] **ID:54/Query/Camera2/Clahe/Mode** - Query clahe mode from camera 2.
[system -> client] **ID:54/Reply/Camera2/Clahe/Mode:1** - Reply about clahe mode of camera 2.
[client -> system] **ID:55/Query/Camera2/Clahe/Type** - Query clahe type from camera 2.
[system -> client] **ID:55/Reply/Camera2/Clahe/Type:1** - Reply about clahe type from camera 2.

4 2 13 MotionMagnificator control messages

MotionMagnificator is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules. **MotionMagnificator** submodule is responsible for motion amplification filtering.

Table 4.2.13.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/MotionMagnificator/...** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
On	Enable motion magnification.
Off	Disable motion magnification.
Mode:n	Set motion magnification mode. Parameter n (integer): 0 - disabled, 1 - enabled.
Level:n	Set motion magnification level. Parameter n (integer): magnification level [1-100%].

Table 4.2.13.2 - **MotionMagnificator** submodule queries. Messages start from **Query/Camera1(2,3,4)/MotionMagnificator/...** Response from system starts with **Reply/Camera1(2,3,4)/MotionMagnificator/...** Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Mode	Query motion magnification mode. Response: .../MotionMagnificator/Mode:n . Parameter n (integer): 0 - disabled, 1 - enabled.
Level	Query motion magnification type. Response: .../MotionMagnificator/Level:n . Parameter n (integer): magnification level [1-100%].

Some example messages:

[client -> system] **ID:52/Command/Camera1/MotionMagnificator/On** - Enable motion magnification in camera 1.
[system -> client] **ID:52/Ack** - Command accepted.
[client -> system] **ID:53/Command/Camera1/MotionMagnificator/Level:10** - Set motion magnification level in camera 1.
[system -> client] **ID:53/Ack** - Command accepted.
[client -> system] **ID:54/Query/Camera2/MotionMagnificator/Mode** - Query motion magnification mode from camera 2.

[system -> client] **ID:54/Reply/Camera2/MotionMagnificator/Mode:1** - Reply about motion magnification mode of camera 2.

[client -> system] **ID:55/Query/Camera2/MotionMagnificator/Level** - Query motion magnification level from camera 2.

[system -> client] **ID:55/Reply/Camera2/MotionMagnificator/Level:10** - Reply about motion magnification level from camera 2.

4 2 14 VideoStream control messages

Each system has two video servers for each camera: **VideoStream1** and **VideoStream2**. **VideoStream1** and **VideoStream2** are submodules which are part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules.

Table 4.2.14.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/VideoStream1(2)/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Restart	Restart video stream.
On	Enable video stream. NOTE: Service WEB page takes first video stream. When first video stream disabled the WEB page can't display video.
Off	Disable video stream. NOTE: Service WEB page takes first video stream. When first video stream disabled the WEB page can't display video.
Mode:n	Enable/disable video stream. Parameter n (integer): 0 - Disable, 1 - Enable. NOTE: Service WEB page takes first video stream. When first video stream disabled the WEB page can't display video.
RtspPort:n	Set RTSP server port. Parameter n (integer): port from 1024 to 65535 . NOTE: RTSP port will be changed for all video streams for all cameras. After changing RTSP server port all video servers will be restarted. Port value must not conflict with PelcoD TCP/UDP ports, External UP TCP/UDP ports or reserved internal port 59111 , otherwise the command will be rejected (Nac).
RtspMulticastIp:string	Set multicast IP. Parameter string (string): IP. If set to "no" it will be unicast by default. NOTE: RTSP server multicast IP will be changed for all video streams for all cameras. After changing RTSP server multicast IP all video servers will be restarted.
RtspMulticastPort:n	Set multicast port. Parameter n (integer): multicast port. NOTE: RTSP server multicast port will be changed for all video streams for all cameras. After changing RTSP server multicast port all video servers will be restarted.
RtmpMode:n	Enable/disable RTMP server for all streamers. Parameter n (integer): 0 - Disable, 1 - Enable.

Command	Description
RtmpPort:n	Set RTMP server port. Parameter n (integer): port from 1024 to 65535 . NOTE: RTMP port will be changed for all video streams for all cameras. After changing RTMP server port all video servers will be restarted. Port value must not conflict with PelcoD TCP/UDP ports, External UP TCP/UDP ports or reserved internal port 59111 , otherwise the command will be rejected (Nac).
HlsMode:n	Enable/disable HLS server for all streamers. Parameter n (integer): 0 - Disable, 1 - Enable.
HlsPort:n	Set HLS server port. Parameter n (integer): port from 1024 to 65535 . NOTE: HLS port will be changed for all video streams for all cameras. After changing HLS server port all video servers will be restarted. Port value must not conflict with PelcoD TCP/UDP ports, External UP TCP/UDP ports or reserved internal port 59111 , otherwise the command will be rejected (Nac).
SrtMode:n	Enable/disable SRT server for all streamers. Parameter n (integer): 0 - Disable, 1 - Enable.
SrtPort:n	Set SRT server port. Parameter n (integer): port from 1024 to 65535 . NOTE: SRT port will be changed for all video streams for all cameras. After changing SRT server port all video servers will be restarted. Port value must not conflict with PelcoD TCP/UDP ports, External UP TCP/UDP ports or reserved internal port 59111 , otherwise the command will be rejected (Nac).
RtpMode:n	Enable/disable RTP direct streaming. Parameter n (integer): 0 - Disable, 1 - Enable.
RtpPort:n	Set RTP direct streaming port. Parameter n (integer): port from 1024 to 65535 .
Resolution:WxH	Set video stream resolution. Parameter WxH (integers): combination of width (W) and height (H), pixels. Video server has input video from camera in original camera resolution. If video stream resolution differs from camera resolution, the video server will scale video to fit stream resolution. Scale algorithm depends on FitMode parameter. NOTE: the NexOS supports only particular resolutions: 1920x1080, 1280x720, 1280x1024, 720x576, 720x486, 704x480, 640x480, 640x512, 704x240, 640x360, 352x288 and 320x240 .
User:string	Set RTSP user name. Parameter string (string): user name, no - no user name. NOTE: RTSP server user name will be changed for all video streams for all cameras. After changing RTSP server user name all video servers will be restarted.
Password:string	Set RTSP password. Parameter string (string): password, no - no password. NOTE: RTSP server password will be changed for all video streams for all cameras. After changing RTSP server password all video servers will be restarted.

Command	Description
Ip:string	Set destination IP address for the RTP stream. Parameter string (string): IP address.
Suffix:string	Set RTSP stream name. Parameter string (string): stream name. For example, if the suffix is live , the RTSP URL will be rtsp://IP:port/live . Note: suffix must be unique among all VideoStream1(2) of all cameras, otherwise the command will be rejected (Nac).
MinBitrateKbps:n	Set minimum bitrate, kilobits per second. Parameter n (integer): bitrate [kbps].
MaxBitrateKbps:n	Set maximum bitrate, kilobits per second. Parameter n (integer): bitrate [kbps].
BitrateKbps:n	Set current bitrate, kilobits per second. Parameter n (integer): bitrate [kbps].
BitrateMode:n	Set bitrate mode. Parameter n (integer): 0 - Constant bitrate, 1 - Variable bitrate.
Fps:n	Set FPS. Parameter n (integer): FPS. Max 30 FPS.
Gop:n	Set GOP size (period of key frames for H264 and H265 codecs). Parameter n (integer): GOP size.
H264Profile:n	Set H264 code profile. Parameter n (integer): 0 - Baseline, 1 - Main, 2 - High.
Codec:string	Set stream codec. Parameter string (string): H264 - H264 codec, H265 - H265 (HEVC) codec, JPEG - JPEG.
JpegQuality:n	Set JPEG quality when codec is set to JPEG. Parameter n (integer): quality value.
FitMode:n	Set video scaling mode. Video server has input video from camera in original camera resolution. If video stream resolution differs from camera resolution, the video server will scale video to fit stream resolution. Parameter n (integer): 0 - Fit, 1 - Crop.
OverlayMode:n	Enable/disable video overlay. Parameter n (integer): 0 - Disable, 1 - Enable.
MetadataMode:n	Enable/disable metadata streaming. Parameter n (integer): 0 - Disable, 1 - Enable.

Command	Description
MetadataSuffix:string	RTP media track name for metadata within the same RTSP stream. Parameter string (string): SMPTE336M - for KLV metadata, VND.ONVIF.METADATA - for ONVIF metadata (not used currently). Only two values are allowed: "SMPTE336M" (for KLV metadata per SMPTE 336M standard) and "VND.ONVIF.METADATA" (for ONVIF metadata). Default: "SMPTE336M" . This value is used as the <code>rtmpmap</code> attribute name for the secondary <code>m=application</code> media line (payload type 98 , clock rate 90000 Hz) in the SDP descriptor. The metadata track is muxed with the video track in the same stream path — it is not a separate RTSP endpoint. The resulting SDP contains two media lines: <code>m=video</code> for the video codec and <code>m=application</code> for the metadata track named by this parameter. If an invalid value is provided to <code>initVStreamer</code> , it will be silently corrected to "SMPTE336M" . If an invalid value is provided to <code>setParam</code> , the method will return FALSE. If changed, the mediamtx process will be restarted by the library automatically.
Custom1:n	Set UDP max payload size for video server configuration (100..65535 bytes). Default: 1472 . Parameter n (integer): UDP max payload size [100:65535].

Table 4.2.14.2 - Query commands. Messages start from **Query/Camera1(2,3,4)/VideoStream1(2)/....** Response from system starts with **Reply/Camera1(2,3,4)/VideoStream1(2)/....** Expected response time: **IP ping time to the system + 5 msec**. For some internal and combined-command scenarios the same parameters can also be addressed via **.../Query/Camera1(2,3,4)/Vpp/VideoStream1(2)/....**

Command	Description
Mode	Query stream mode (enable/disable). Response: .../VideoStream1(2)/Mode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
Type	Query stream type. Response: .../VideoStream1(2)/Type:n . Parameter n (integer): internal stream type identifier.
RtspPort	Query RTSP server port. Response: .../VideoStream1(2)/RtspPort:n . Parameter n (integer): port from 1024 to 65535 .
RtspMulticastIp	Query multicast IP. Response: .../VideoStream1(2)/RtspMulticastIp:string . Parameter string (string): IP. If set to "no" it will be unicast by default.
RtspMulticastPort	Query multicast port. Response: .../VideoStream1(2)/RtspMulticastPort:n . Parameter n (integer): multicast port.
RtmpMode	Query RTMP server mode. Response: .../VideoStream1(2)/RtmpMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
RtmpPort	Query RTMP server port. Response: .../VideoStream1(2)/RtmpPort:n . Parameter n (integer): port from 1024 to 65535 .
HlsMode	Query HLS server mode. Response: .../VideoStream1(2)/HlsMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.

Command	Description
HlsPort	Query HLS server port. Response: .../VideoStream1(2)/HlsPort:n . Parameter n (integer): port from 1024 to 65535 .
SrtMode	Query SRT server mode. Response: .../VideoStream1(2)/SrtMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
SrtPort	Query SRT server port. Response: .../VideoStream1(2)/SrtPort:n . Parameter n (integer): port from 1024 to 65535 .
Resolution	Query video stream resolution. Response: .../VideoStream1(2)/Resolution:WxH . Parameter WxH (integers): combination of width (W) and height (H), pixels. Video server has input video from camera in original camera resolution. If video stream resolution differs from camera resolution, the video server will scale video to fit stream resolution. Scale algorithm depends on FitMode parameter.
User	Query RTSP user name. Response: .../VideoStream1(2)/User:string . Parameter string (string): user name, no - no user name.
Password	Query RTSP password. Response: .../VideoStream1(2)/Password:string . Parameter string (string): password, no - no password.
Suffix	Query RTSP stream name. Response: .../VideoStream1(2)/Suffix:string . Parameter string (string): stream name. For example, live and in this case RTSP string will be rtsp://IP:port/live .
MinBitrateKbps	Query minimum bitrate, kilobits per second. Response: .../VideoStream1(2)/MinBitrateKbps:n . Parameter n (integer): bitrate [kbps].
MaxBitrateKbps	Query maximum bitrate, kilobits per second. Response: .../VideoStream1(2)/MaxBitrateKbps:n . Parameter n (integer): bitrate [kbps].
BitrateKbps	Query current bitrate, kilobits per second. Response: .../VideoStream1(2)/BitrateKbps:n . Parameter n (integer): bitrate [kbps].
BitrateMode	Query bitrate mode. Response: .../VideoStream1(2)/BitrateMode:n . Parameter n (integer): 0 - Constant bitrate, 1 - Variable bitrate.
Fps	Query FPS. Response: .../VideoStream1(2)/Fps:n . Parameter n (integer): FPS.
Gop	Query GOP size (period of key frames for H264 and H265 codecs). Response: .../VideoStream1(2)/Gop:n . Parameter n (integer): GOP size.
H264Profile	Query H264 code profile. Response: .../VideoStream1(2)/H264Profile:n . Parameter n (integer): 0 - Baseline, 1 - Main, 2 - High.
Codec	Query stream codec. Response: .../VideoStream1(2)/Codec:string . Parameter string (string): H264 - H264 codec, H265 - H265 (HEVC) codec, JPEG - JPEG.
FitMode	Query video scaling mode. Video server has input video from camera in original camera resolution. If video stream resolution differs from camera resolution, the video server will scale video to fit stream resolution. Response: .../VideoStream1(2)/FitMode:n . Parameter n (integer): 0 - Fit, 1 - Crop.

Command	Description
OverlayMode	Query video overlay mode. Response: .../VideoStream1(2)/OverlayMode:n . Parameter n : 0 - Disable, 1 - Enable.
Ip	Query destination IP address configured for RTP video stream. Response: .../VideoStream1(2)/Ip:string . Parameter string (string): IP address.
RtpPort	Query RTP direct streaming port. Response: .../VideoStream1(2)/RtpPort:n . Parameter n (integer): port.
RtpMode	Query RTP direct streaming mode. Response: .../VideoStream1(2)/RtpMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
IP	Only via Vpp prefix. Query host IP used to build streaming URLs. Messages start from .../Query/Camera1(2,3,4)/Vpp/VideoStream1(2)/IP . Response: .../Vpp/VideoStream1(2)/IP:string . Parameter string (string): IP address of the NexOS device.
WebRtcPort	Only via Vpp prefix. Query WebRTC server port. Messages start from .../Query/Camera1(2,3,4)/Vpp/VideoStream1(2)/WebRtcPort . Response: .../Vpp/VideoStream1(2)/WebRtcPort:n . Parameter n (integer): port from 1024 to 65535 .
MetadataMode	Query metadata streaming mode. Response: .../VideoStream1(2)/MetadataMode:n . Parameter n : 0 - Disable, 1 - Enable.
MetadataSuffix	Query RTP media track name for metadata within the same RTSP stream. Response: .../VideoStream1(2)/MetadataSuffix:string . Parameter string (string): SMPTE336M - for KLV metadata, VND.ONVIF.METADATA - for ONVIF metadata (not used currently). Only two values are allowed: "SMPTE336M" (for KLV metadata per SMPTE 336M standard) and "VND.ONVIF.METADATA" (for ONVIF metadata). Default: "SMPTE336M" . This value is used as the <code>rtmap</code> attribute name for the secondary <code>m=application</code> media line (payload type 98 , clock rate 90000 Hz) in the SDP descriptor. The metadata track is muxed with the video track in the same stream path — it is not a separate RTSP endpoint. The resulting SDP contains two media lines: <code>m=video</code> for the video codec and <code>m=application</code> for the metadata track named by this parameter. If an invalid value is provided to <code>initvstreamer</code> , it will be silently corrected to "SMPTE336M" . If an invalid value is provided to <code>setParam</code> , the method will return FALSE. If changed, the mediamtx process will be restarted by the library automatically.
Custom1	Query UDP max payload size for video server configuration (100..65535 bytes). Default: 1472 . Response: .../VideoStream1(2)/Custom1:n . Parameter n (integer): UDP max payload size [100:65535].

Some example messages:

[client -> system] **ID:68/Command/Camera1/VideoStream1/On** - Enable first video server of camera 1.

[system -> client] **ID:68/Ack** - Command accepted.

[client -> system] **ID:69/Command/Camera1/VideoStream1/RtspMulticastPort:7031** - Set RTSP multicast port of first video server of camera 1.

[system -> client] **ID:69/Ack** - Command accepted.

[client -> system] **ID:70/Query/Camera1/VideoStream1/Mode** - Query first video server mode of camera 1.
 [system -> client] **ID:70/Reply/Camera1/VideoStream1/Mode:1** - Reply first video server mode of camera 1.
 [client -> system] **ID:71/Query/Camera1/VideoStream1/BitrateKbps** - Query bitrate of first video server mode of camera 1.
 [system -> client] **ID:71/Reply/Camera1/VideoStream1/BitrateKbps:5000** - Reply about bitrate of first video server mode of camera 1.

4 2 15 Overlay control messages

Each camera has a video overlay engine. The video overlay engine puts necessary information on video. The engine is the same for all video servers for one camera, including streaming to the service WEB page (service WEB page takes video from first video stream). **Overlay** is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules.

Table 4.2.15.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/Overlay/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
CrossHairMode:n	Set cross hair display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
CrossHairSize:n	Set cross hair size, pixel. Parameter n (integer): size, pixels (1 - 100).
CrossHairColor:n	Set cross hair color. Parameter n (integer): 0 - white, 1 - black, 2 - red, 3 - green, 4 - blue, 5 - contrast.
DateTimeMode:n	Set date and time display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
ZoomPosMode:n	Set zoom position display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
FocusMode:n	Set focus mode display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
LrfMode:n	Set Lrf data display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
DigitalZoomMode:n	Set digital zoom display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
ClaheMode:n	Set clahe display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
TrackerMode:n	Set tracker display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
PanTiltPosMode:n	Set pan-tilt position display mode. Parameter n (integer): 0 - Disable, 1 - Enable.

Command	Description
MotionDetectorMode:n	Set motion detector results display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
ChangesDetectorMode:n	Set changes detector results display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
ClassificationMode:n	Set classification results display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
FontSize:n	Set overlay font size, %. Parameter n (integer): font size 1-99%.
MotionMagnificatorMode:n	Set motion magnification display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
CompassBearingMode:n	Set compass bearing display mode. Parameter n (integer): 0 - Disable, 1 - Enable.

Table 4.2.15.2 - Query commands. Messages start from **Query/Camera1(2,3,4)/Overlay/....** Response from system starts with **Reply/Camera1(2,3,4)/Overlay/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
CrossHairMode	Query cross hair display mode. Response: .../Overlay/CrossHairMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
CrossHairSize	Query cross hair size, pixels. Response: .../Overlay/CrossHairSize:n . Parameter n (integer): size, pixels (1 - 100).
CrossHairColor	Query cross hair color. Response: .../Overlay/CrossHairColor:n . Parameter n (integer): 0 - white, 1 - black, 2 - red, 3 - green, 4 - blue, 5 - contrast.
DateTimeMode	Query date and time display mode. Response: .../Overlay/DateTimeMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
ZoomPosMode	Query zoom position display mode. Response: .../Overlay/ZoomPosMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
FocusMode	Query focus mode display mode. Response: .../Overlay/FocusMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
LrfMode	Query laser range finder data display mode. Response: .../Overlay/LrfMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
DigitalZoomMode	Query digital zoom display mode. Response: .../Overlay/DigitalZoomMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.

Command	Description
ClaheMode	Query clahe display mode. Response: .../Overlay/ClaheMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
TrackerMode	Query tracker display mode. Response: .../Overlay/TrackerMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
PanTiltPosMode	Query pan-tilt position display mode. Response: .../Overlay/PanTiltPosMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
MotionDetectorMode	Query motion detector results display mode. Response: .../Overlay/MotionDetectorMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
ChangesDetectorMode	Query changes detector results display mode. Response: .../Overlay/ChangesDetectorMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
ClassificationMode	Query classification results display mode. Response: .../Overlay/ClassificationMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
FontSize	Query overlay font size, %. Response: .../Overlay/FontSize:n . Parameter n (integer): font size 1-99%.
MotionMagnificatorMode	Query motion magnification display mode. Response: .../Overlay/MotionMagnificatorMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
CompassBearingMode	Query compass bearing display mode. Response: .../Overlay/CompassBearingMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.

Some example messages:

[client -> system] **ID:72/Command/Camera1/Overlay/CrossHairMode:1** - Enable cross hair display for camera 1.

[system -> client] **ID:72/Ack** - Command accepted.

[client -> system] **ID:73/Command/Camera1/Overlay/CrossHairSize:40** - Set cross hair size for camera 1.

[system -> client] **ID:73/Ack** - Command accepted.

[client -> system] **ID:74/Query/Camera1/Overlay/LrfMode** - Query laser range finder display mode from camera 1.

[system -> client] **ID:74/Reply/Camera1/Overlay/LrfMode:1** - Reply about laser range finder display mode from camera 1.

[client -> system] **ID:75/Query/Camera1/Overlay/FontSize** - Query overlay font size from camera 1.

[system -> client] **ID:75/Reply/Camera1/Overlay/FontSize:60** - Reply about overlay font size from camera 1.

4 2 16 Web RTC Overlay control messages

Each camera has a video overlay engine. The video overlay engine puts necessary information on video. The engine is the same for all video servers for one camera, including streaming to the service WEB page (service WEB page takes video from first video stream). **WebRtcOverlay** is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules.

Table 4.2.16.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/WebRtcOverlay/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
OverlayMode:n	Set overlay visibility. Parameter n (integer): 0 - Disable, 1 - Enable.
CrossHairMode:n	Set cross hair display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
CrossHairSize:n	Set cross hair size, pixel. Parameter n (integer): size, pixels (1 - 100).
CrossHairColor:n	Set cross hair color. Parameter n (integer): 0 - white, 1 - black, 2 - red, 3 - green, 4 - blue, 5 - contrast.
DateTimeMode:n	Set date and time display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
ZoomPosMode:n	Set zoom position display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
FocusMode:n	Set focus mode display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
LrfMode:n	Set Lrf data display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
DigitalZoomMode:n	Set digital zoom display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
ClaheMode:n	Set clahe display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
TrackerMode:n	Set tracker display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
PanTiltPosMode:n	Set pan-tilt position display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
MotionDetectorMode:n	Set motion detector results display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
ChangesDetectorMode:n	Set changes detector results display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
ClassificationMode:n	Set classification results display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
FontSize:n	Set overlay font size, %. Parameter n (integer): font size 1-99%.

Command	Description
MotionMagnificatorMode:n	Set motion magnification display mode. Parameter n (integer): 0 - Disable, 1 - Enable.
CompassBearingMode:n	Set compass bearing display mode. Parameter n (integer): 0 - Disable, 1 - Enable.

Table 4.2.16.2 - Query commands. Messages start from **Query/Camera1(2,3,4)/WebRtcOverlay/...**
Response from system starts with **Reply/Camera1(2,3,4)/WebRtcOverlay/...** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
OverlayMode	Query overlay visibility. Response: .../WebRtcOverlay/OverlayMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
CrossHairMode	Query cross hair display mode. Response: .../WebRtcOverlay/CrossHairMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
CrossHairSize	Query cross hair size, pixels. Response: .../WebRtcOverlay/CrossHairSize:n . Parameter n (integer): size, pixels (1 - 100).
CrossHairColor	Query cross hair color. Response: .../WebRtcOverlay/CrossHairColor:n . Parameter n (integer): 0 - white, 1 - black, 2 - red, 3 - green, 4 - blue, 5 - contrast.
DateTimeMode	Query date and time display mode. Response: .../WebRtcOverlay/DateTimeMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
ZoomPosMode	Query zoom position display mode. Response: .../WebRtcOverlay/ZoomPosMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
FocusMode	Query focus mode display mode. Response: .../WebRtcOverlay/FocusMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
LrfMode	Query laser range finder data display mode. Response: .../WebRtcOverlay/LrfMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
DigitalZoomMode	Query digital zoom display mode. Response: .../WebRtcOverlay/DigitalZoomMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
ClaheMode	Query clahe display mode. Response: .../WebRtcOverlay/ClaheMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.

Command	Description
TrackerMode	Query tracker display mode. Response: .../WebRtcOverlay/TrackerMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
PanTiltPosMode	Query pan-tilt position display mode. Response: .../WebRtcOverlay/PanTiltPosMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
MotionDetectorMode	Query motion detector results display mode. Response: .../WebRtcOverlay/MotionDetectorMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
ChangesDetectorMode	Query changes detector results display mode. Response: .../WebRtcOverlay/ChangesDetectorMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
ClassificationMode	Query classification results display mode. Response: .../WebRtcOverlay/ClassificationMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
FontSize	Query overlay font size, %. Response: .../WebRtcOverlay/FontSize:n . Parameter n (integer): font size 1-99%.
MotionMagnificatorMode	Query motion magnification display mode. Response: .../WebRtcOverlay/MotionMagnificatorMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.
CompassBearingMode	Query compass bearing display mode. Response: .../WebRtcOverlay/CompassBearingMode:n . Parameter n (integer): 0 - Disable, 1 - Enable.

Some example messages:

[client -> system] **ID:72/Command/Camera1/WebRtcOverlay/CrossHairMode:1** - Enable cross hair display for camera 1.

[system -> client] **ID:72/Ack** - Command accepted.

[client -> system] **ID:73/Command/Camera1/WebRtcOverlay/CrossHairSize:40** - Set cross hair size for camera 1.

[system -> client] **ID:73/Ack** - Command accepted.

[client -> system] **ID:74/Query/Camera1/WebRtcOverlay/LrfMode** - Query laser range finder display mode from camera 1.

[system -> client] **ID:74/Reply/Camera1/WebRtcOverlay/LrfMode:1** - Reply about laser range finder display mode from camera 1.

[client -> system] **ID:75/Query/Camera1/WebRtcOverlay/FontSize** - Query overlay font size from camera 1.

[system -> client] **ID:75/Reply/Camera1/WebRtcOverlay/FontSize:60** - Reply about overlay font size from camera 1.

4 2 17 BadPixelProcessor control messages

BadPixelProcessor functional module is responsible for detection and compensation of dead pixels for LWIR, MWIR, SWIR and daylight cameras. **BadPixelProcessor** is a submodule which is part of **Camera1**, **Camera2**, **Camera3** and **Camera4** modules.

Table 4.2.17.1 - Action commands and commands to change parameters. Messages start from **.../Command/Camera1(2,3,4)/BadPixelProcessor/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Reset	Start bad pixels detection. After this command BadPixelProcessor functional module will detect bad pixels during 2 seconds and after will be able to compensate them in real-time. WARNING: to make successful bad pixels detection it is recommended to make full zoom, defocus image and put camera field of view on flat surface. In this case bad pixels will be well visible.
Mode:n	Enable / disable bad pixel correction. Parameter n (integer): 0 - disable, 1 - enable.
ChangeThreshold:n	Pixel value difference between a bad pixel candidate and its neighbors. Parameter n (integer): pixel brightness value in [0:255] range. The pixel will be classified as a bad pixel if during 60 frames (from the Reset command) the brightness value did not change by more than the ChangeThreshold value. Default value 40 . Any pixel brightness value within [0:255] range.
LowThreshold:n	Low threshold value for bad pixel candidates. Parameter n (integer): pixel brightness value in [0:255] range. The pixel will be classified as a bad pixel candidate if the brightness value is less than the LowThreshold parameter. Default value 20 . Any pixel brightness value within [0:255] range.
HighThreshold:n	High threshold value for bad pixel candidates. Parameter n (integer): pixel brightness value in [0:255] range. The pixel will be classified as a bad pixel candidate if the brightness value is more than the HighThreshold parameter. Default value 235 . Any pixel brightness value within [0:255] range.

Table 4.2.17.2 - Query commands. Messages start from **Query/Camera1(2,3,4)/BadPixelProcessor/....** Response from system starts with **Reply/Camera1(2,3,4)/BadPixelProcessor/....** Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Mode	Query enable status of the bad pixel correction. Response: .../BadPixelProcessor/Mode:n . Parameter n (integer): 0 - disable, 1 - enable.
ChangeThreshold	Query pixel value difference between a bad pixel candidate and its neighbors. Response: .../BadPixelProcessor/ChangeThreshold:n . Parameter n (integer): pixel brightness value in [0:255] range. The pixel will be classified as a bad pixel if during 60 frames (from the Reset command) the brightness value did not change by more than the ChangeThreshold value. Default value 40 . Any pixel brightness value within [0:255] range.

Command	Description
LowThreshold	Query low threshold value for bad pixel candidates. Response: .../BadPixelProcessor/LowThreshold:n . Parameter n (integer): pixel brightness value in [0:255] range. The pixel will be classified as a bad pixel candidate if the brightness value is less than the LowThreshold parameter. Default value 20 . Any pixel brightness value within [0:255] range.
HighThreshold	Query high threshold value for bad pixel candidates. Response: .../BadPixelProcessor/HighThreshold:n . Parameter n (integer): pixel brightness value in [0:255] range. The pixel will be classified as a bad pixel candidate if the brightness value is more than the HighThreshold parameter. Default value 235 . Any pixel brightness value within [0:255] range.

Some example messages:

[client -> system] **ID:72/Command/Camera1/BadPixelProcessor/Mode:1** - Enable bad pixel correction.

[system -> client] **ID:72/Ack** - Command accepted.

[client -> system] **ID:73/Command/Camera1/BadPixelProcessor/ChangeThreshold:40** - Set change threshold.

[system -> client] **ID:73/Ack** - Command accepted.

[client -> system] **ID:74/Query/Camera1/BadPixelProcessor/LowThreshold** - Query low threshold.

[system -> client] **ID:74/Reply/Camera1/BadPixelProcessor/LowThreshold:20** - Reply about low threshold.

[client -> system] **ID:75/Query/Camera1/BadPixelProcessor/HighThreshold** - Query high threshold.

[system -> client] **ID:75/Reply/Camera1/BadPixelProcessor/HighThreshold:235** - Reply about high threshold.

4 3 Companion control messages

Each system includes few companion board services. Each CPU board (main board and payload boards) includes companion board service. Possible services names for control protocol: **Companion1** (companion service for first payload board), **Companion2** (companion service for second payload board), **Companion3** (companion service for third payload board) and **CompanionMain** (companion service for main board). Companion submodule is responsible service function like wiper control, heater control, power control etc.

Table 4.3.1 - Action commands and commands to change parameters. Messages start from **.../Command/Companion1(2,3,Main)/...**. Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**. Central system service sends command to **Companion1(2,3,Main)** service and replies **Ack** if the message was sent without checking if the message is valid.

Command	Supported by	Description
WiperMode:n	Companion(1,2,3)	Set wiper mode. Parameter n (integer): 0 - off, 1 - on.
WiperTimeoutSec:n	Companion(1,2,3)	Set wiper timeout, sec. Parameter n (integer): seconds after start to turn off automatically, 0 - not set (no timeout).

Command	Supported by	Description
HeaterMode:n	Companion(1,2,3)	Set heater mode. Parameter n (integer): 0 - off, 1 - on.
HeaterTimeoutSec:n	Companion(1,2,3)	Set heater timeout, sec. Parameter n (integer): seconds after start to turn off automatically, 0 - not set (no timeout).
PowerChannel2Mode:n	Companion(1,2,3)	Set power channel 2 mode (lens and camera). Parameter n (integer): 0 - off, 1 - on.
SlgStrobeTimeoutMsec:n	Companion(1,2,3)	Set signal light gun strobe timeout. Parameter n (integer): strobe timeout [msec].

Table 4.3.2 - Companion1(2,3,Main) submodule queries. Messages start from **Query/Companion1(2,3,Main)/....** Response from system starts with **Reply/Companion1(2,3,Main)/....** Central control service sends message to **Companion1(2,3,Main)** service. Expected response time: **IP ping time to the system + 5 msec** if central control service has received a valid reply from **Companion1(2,3,Main)** service, or **IP ping time to the system + 105 msec** if no valid reply from **Companion1(2,3,Main)** service.

Command	Supported by	Description
IsOpen	Companion(1,2,3,Main)	Query initialization status. Response: .../Companion1(2,3,Main)/IsOpen:n . Parameter n (integer): 0 - not initialized, 1 - initialized.
IsConnected	Companion(1,2,3,Main)	Query connection status. Response: .../Companion1(2,3,Main)/IsConnected:n . Parameter n (integer): 0 - no response from hardware, 1 - connected.
WiperMode	Companion(1,2,3)	Query wiper mode. Response: .../Companion1(2,3,Main)/WiperMode:n . Parameter n (integer): 0 - off, 1 - on.
WiperTimeoutSec	Companion(1,2,3)	Query wiper timeout. Response: .../Companion1(2,3,Main)/WiperTimeoutSec:n . Parameter n (integer): seconds after start to turn off automatically, 0 - not set (no timeout).
HeaterMode	Companion(1,2,3,Main)	Query heater mode. Response: .../Companion1(2,3,Main)/HeaterMode:n . Parameter n (integer): 0 - off, 1 - on.
HeaterTimeoutSec	Companion(1,2,3)	Query heater timeout. Response: .../Companion1(2,3,Main)/HeaterTimeoutSec:n . Parameter n (integer): seconds after start to turn off automatically, 0 - not set (no timeout).
SlgStrobeTimeoutMsec	Companion(1,2,3)	Query signal light gun strobe timeout. Response: .../Companion1(2,3,Main)/SlgStrobeTimeoutMsec:n . Parameter n (integer): strobe timeout [msec] range: 0-50ms
PowerChannel1Mode	Companion(1,2,3)	Query power channel 1 mode. Response: .../Companion1(2,3,Main)/PowerChannel1Mode:n . Parameter n (integer): 0 - off, 1 - on.
PowerChannel2Mode	Companion(1,2,3)	Query power channel 2 mode. Response: .../Companion1(2,3,Main)/PowerChannel2Mode:n . Parameter n (integer): 0 - off, 1 - on.
PowerChannel3Mode	CompanionMain	Query power channel 3 mode. Response: .../Companion1(2,3,Main)/PowerChannel3Mode:n . Parameter n (integer): 0 - off, 1 - on.
PowerChannel1Voltage	Companion(1,2,3,Main)	Query power channel 1 voltage. Response: .../Companion1(2,3,Main)/PowerChannel1Voltage:n . Parameter n (float): voltage.
PowerChannel2Voltage	Companion(1,2,3,Main)	Query power channel 2 voltage. Response: .../Companion1(2,3,Main)/PowerChannel2Voltage:n . Parameter n (float): voltage.
PowerChannel3Voltage	CompanionMain	Query power channel 3 voltage. Response: .../Companion1(2,3,Main)/PowerChannel3Voltage:n . Parameter n (float): voltage.

Command	Supported by	Description
PowerChannel1Current	Companion(1,2,3,Main)	Query power channel 1 current. Response: .../Companion1(2,3,Main)/PowerChannel1Current:n. Parameter n (float): current.
PowerChannel2Current	CompanionMain(1,2,3,Main)	Query power channel 2 current. Response: .../Companion1(2,3,Main)/PowerChannel2Current:n. Parameter n (float): current.
PowerChannel3Current	CompanionMain	Query power channel 3 current. Response: .../Companion1(2,3,Main)/PowerChannel3Current:n. Parameter n (float): current.
PowerChannel1Temperature	Companion(1,2,3,Main)	Query power channel 1 temperature, degree. Response: .../Companion1(2,3,Main)/PowerChannel1Temperature:n Parameter n (float): temperature [deg].
PowerChannel2Temperature	Companion(1,2,3,Main)	Query power channel 2 temperature, degree. Response: .../Companion1(2,3,Main)/PowerChannel2Temperature:n. Parameter n (float): temperature [deg].
PowerChannel3Temperature	CompanionMain	Query power channel 3 temperature, degree. Response: .../Companion1(2,3,Main)/PowerChannel3Temperature:n. Parameter n (float): temperature [deg].
MainTemperature	Companion(1,2,3,Main)	Query main temperature, degree. Response: .../Companion1(2,3,Main)/MainTemperature:n. Parameter n (float): temperature [deg].
MainHumidity	Companion(1,2,3,Main)	Query main humidity. Response: .../Companion1(2,3,Main)/MainHumidity:n. Parameter n (float): humidity [%].

Some example messages:

[client -> system] **ID:56/Command/Companion1/WiperMode:1** - Enable wiper in payload board 1.

[system -> client] **ID:56/Ack** - Command accepted.

[client -> system] **ID:57/Command/Companion1/HeaterMode:1** - Enable heater in payload board 1.

[system -> client] **ID:57/Ack** - Command accepted.

[client -> system] **ID:58/Query/Companion2/PowerChannel1Mode** - Query power channel 1 mode from payload board 2.

[system -> client] **ID:58/Reply/Companion2/PowerChannel1Mode:1** - Reply about power channel 1 mode of payload board 2.

[client -> system] **ID:59/Query/CompanionMain/MainTemperature** - Query main temperature from main board.

[system -> client] **ID:59/Reply/CompanionMain/MainTemperature:56** - Reply about main temperature from main board.

4 4 Lrf control messages

Lrf (laser range finder) is an independent submodule. **Lrf** submodule represents a laser range finder. The system can have only one laser range finder. Laser range finder is connected to one of the payloads.

Table 4.4.1 - Action commands and commands to change parameters. Messages start from **.../Command/Lrf/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**. Central system service sends command to **Lrf** service and replies **Ack** if the message was sent without checking if the message is valid.

Command	Description
Arm	Arm laser pointer.
Disarm	Disarm laser pointer.
Measure	Measure distance once.

Command	Description
LowPowerMode:n	Set low power mode. NOTE: to be able use pointer and measurement the low power mode must be off. Parameter n (integer): 0 - Off, 1 - Lowest power, 2 , ... 6 - Highest power.
PointerMode:n	Set pointer mode (Continuous laser emission). Parameter n (integer): 0 - Off, 1 - On.
PointerModeTimeoutSec:n	Set pointer mode timeout, sec. Parameter n (integer): from 1 to 20 sec.
ArmMode:n	Set arm mode (arm/disarm laser pointer). Parameter n (integer): 0 - Disarmed, 1 - Armed.
OperatingMode:n	Set operating mode. Parameter n (integer): 0 - Off (don't use because it will turn off LRF), 1 - Stand-by-mode, 2 - Normal mode.
ContinuousMeasuringMode:n	Set continuous measuring mode. Parameter n (integer): 0 - STOP, 1 - Repetition rate of 0.5 Hz, 2 - Repetition rate of 1 Hz, 3 - Repetition rate of 3 Hz, 4 - Repetition rate of 5 Hz. 5 - Repetition rate of 10 Hz.
ContinuousModeTimeoutSec:n	Set continuous measuring mode timeout, sec. Parameter n (integer): timeout [sec], 0 - Off.
MinGateDistance:n	Set minimal gate distance, meters. Measured distance less than this value will be ignored. Parameter n (integer): distance [meters].
MaxGateDistance:n	Set maximum gate distance, meters. Measured distance more than this value will be ignored. Parameter n (integer): distance [meters].

Table 4.4.2 - Query commands. Messages start from **Query/Lrf/....** Response from system starts with **Reply/Lrf/....** Central control service sends message to **Lrf** service. Expected response time: **IP ping time to the system + 5 msec** if central control service has received a valid reply from **Lrf** service, or **IP ping time to the system + 105 msec** if no valid reply from **Lrf** service.

Command	Description
IsOpen	Query initialization status. Response: Reply/Lrf/IsOpen:n . Parameter n (integer): 0 - not initialized, 1 - initialized.
IsConnected	Query connection status. Response: Reply/Lrf/IsConnected:n . Parameter n (integer): 0 - no response from hardware, 1 - connected.
LowPowerMode	Query low power mode. Response: Reply/Lrf/LowPowerMode:n . Parameter n (integer): 0 - Off, 1 - Lowest power, 2 , ... 6 - Highest power.
PointerMode	Query pointer mode. Response: Reply/Lrf/PointerMode:n . Parameter n (integer): 0 - Off, 1 - On.

Command	Description
PointerModeTimeoutSec	Query pointer mode timeout, sec. Response: Reply/Lrf/PointerModeTimeoutSec:n . Parameter n (integer): timeout [1-20] sec.
ArmMode	Query pointer arm mode. Response: Reply/Lrf/ArmMode:n . Parameter n (integer): 0 - Disarmed, 1 - Armed.
OperatingMode	Query operating mode. Response: Reply/Lrf/OperatingMode:n . Parameter n (integer): 0 - Off (don't use because it will turn off LRF), 1 - Stand-by-mode, 2 - Normal mode.
ContinuousMeasuringMode	Query continuous measuring mode. Response: Reply/Lrf/ContinuousMeasuringMode:n . Parameter n (integer): 0 - STOP, 1 - Repetition rate of 0.5 Hz, 2 - Repetition rate of 1 Hz, 3 - Repetition rate of 3 Hz, 4 - Repetition rate of 5 Hz. 5 - Repetition rate of 10 Hz.
ContinuousModeTimeoutSec	Query continuous measuring mode timeout, sec. Response: Reply/Lrf/ContinuousModeTimeoutSec:n . Parameter n (integer): 0 - Off.
MinGateDistance	Query minimal gate distance, meters. Measured distance less than this value will be ignored. Response: Reply/Lrf/MinGateDistance:n . Parameter n (integer): distance [meters].
MaxGateDistance	Query maximum gate distance, meters. Measured distance more than this value will be ignored. Response: Reply/Lrf/MaxGateDistance:n . Parameter n (integer): distance [meters].
Distance	Query current measured distance, meters. Response: Reply/Lrf/Distance:n . Parameter n (float): distance [meters].

Some example messages:

[client -> system] **ID:64/Command/Lrf/PointerMode:1** - Enable laser pointer.

[system -> client] **ID:64/Ack** - Command accepted.

[client -> system] **ID:65/Command/Lrf/Measure** - Measure distance once.

[system -> client] **ID:65/Ack** - Command accepted.

[client -> system] **ID:66/Query/Lrf/PointerModeTimeoutSec** - Query pointer mode timeout.

[system -> client] **ID:66/Reply/Lrf/PointerModeTimeoutSec:10** - Reply about pointer mode timeout.

[client -> system] **ID:67/Query/Lrf/Distance** - Query measured distance.

[system -> client] **ID:67/Reply/Lrf/Distance:56** - Reply about measured distance.

4 5 Speaker control messages

Speaker is an independent submodule. **Speaker** submodule represents a speaker. The system can have only one speaker. Speaker is connected to main board.

Table 4.5.1 - Action commands and commands to change parameters. Messages start from **.../Command/Speaker/...**. Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**. Central system service sends command to **Speaker** service and replies **Ack** if the message was sent without checking if the message is valid.

Command	Description
Play:name	Start playing audio clip. Parameter name (string): clip name (for example clip1.mp3). NOTE: the audio clips must be uploaded in advance through system WEB interface.
Stop	Stop playing.
Volume:n	Set playback volume. Parameter n (integer): volume [0-100%].

Table 4.5.2 - Query commands. Messages start from **Query/Speaker/...**. Response from system starts with **Reply/Speaker/...**. Central control service sends message to **Speaker** service. Expected response time: **IP ping time to the system + 5 msec** if central control service has received a valid reply from **Speaker** service, or **IP ping time to the system + 105 msec** if no valid reply from **Speaker** service.

Command	Description
Volume	Query playback volume. Response: Reply/Speaker/Volume:n . Parameter n (integer): volume [0-100%].
Mode	Query playback mode. Response: Reply/Speaker/Mode:n . Parameter n (integer): 0 - not playing, 1 - playing.

Some example messages:

[client -> system] **ID:64/Command/Speaker/Play:clip1.mp3** - Play clip1.mp3.

[system -> client] **ID:64/Ack** - Command accepted.

[client -> system] **ID:65/Command/Speaker/Stop** - Stop playback.

[system -> client] **ID:65/Ack** - Command accepted.

[client -> system] **ID:66/Query/Speaker/Volume** - Query playback volume.

[system -> client] **ID:66/Reply/Speaker/Volume:10** - Reply about playback volume (10%).

[client -> system] **ID:67/Command/Speaker/Volume:20** - Set playback volume to 20%.

[system -> client] **ID:67/Ack** - Command accepted.

4 6 General control messages

General is a submodule which is part of main board. **General** submodule is responsible for general system parameters such as slave zoom mode, licensing modules, low latency video streaming, etc.

Table 4.6.1 - Action commands and commands to change parameters. Messages start from **.../Command/General/...**. Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
JafarServiceRestart	Command to restart JafarService.
SlaveZoomMode:n	Slave zoom mode. Parameter n (integer): 0 - Disabled, 1 - Zoom position based synchronization, 2 - FOV (field of view) based synchronization.
SlaveZoomMasterCamera:n	Slave zoom mode master camera. According to zoom position from master camera the system will adjust zoom position of other cameras. Parameter n (integer): 0 - Camera1, 1 - Camera2, 2 - Camera3.

Table 4.6.2 - General submodule queries. Messages start from **Query/General/....** Response from system starts with **Reply/General/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
VideoTrackerToken	Query video tracker license token. The token is used to generate license file for clients. Response: Reply/General/VideoTrackerToken:string . Parameter string (string): license token.
VideoTrackerLicenseStatus	Query video tracker license status. Response: Reply/General/VideoTrackerLicenseStatus:n . Parameter n (integer): 0 - the license is not valid, 1 - the license is valid.
VideoStabiliserToken	Query video stabiliser license token. The token is used to generate license file for clients. Response: Reply/General/VideoStabiliserToken:string . Parameter string (string): license token.
VideoStabiliserLicenseStatus	Query video stabiliser license status. Response: Reply/General/VideoStabiliserLicenseStatus:n . Parameter n (integer): 0 - the license is not valid, 1 - the license is valid.
MotionDetectorToken	Query motion detector license token. The token is used to generate license file for clients. Response: Reply/General/MotionDetectorToken:string . Parameter string (string): license token.
MotionDetectorLicenseStatus	Query motion detector license status. Response: Reply/General/MotionDetectorLicenseStatus:n . Parameter n (integer): 0 - the license is not valid, 1 - the license is valid.
ChangesDetectorToken	Query changes detector license token. The token is used to generate license file for clients. Response: Reply/General/ChangesDetectorToken:string . Parameter string (string): license token.
ChangesDetectorLicenseStatus	Query changes detector license status. Response: Reply/General/ChangesDetectorLicenseStatus:n . Parameter n (integer): 0 - the license is not valid, 1 - the license is valid.
ClassificationToken	Query classification license token. The token is used to generate license file for clients. Response: Reply/General/ClassificationToken:string . Parameter string (string): license token.

Command	Description
ClassificationLicenseStatus	Query classification license status. Response: Reply/General/ClassificationLicenseStatus:n . Parameter n (integer): 0 - the license is not valid, 1 - the license is valid.
SlaveZoomMode	Query slave zoom mode. Response: Reply/General/SlaveZoomMode:n . Parameter n (integer): 0 - Disabled, 1 - Zoom position based synchronization, 2 - FOV (field of view) based synchronization.
SlaveZoomMasterCamera	Query slave zoom mode master camera. Response: Reply/General/SlaveZoomMasterCamera:n . Parameter n (integer): 0 - Camera1, 1 - Camera2, 2 - Camera3.
Mainboard/serial	Query mainboard serial number. Response: Reply/General/Mainboard/serial:string . Parameter string (string): mainboard serial number.
Mainboard/versionNexOS	Query mainboard NexOS version. Response: Reply/General/Mainboard/versionNexOS:string . Parameter string (string): NexOS version string.
Mainboard/trackingEnabled	Query mainboard tracking enabled status. Response: Reply/General/Mainboard/trackingEnabled:s . Parameter s (string): On - tracking enabled, Off - tracking disabled.

Some example messages:

[client -> system] **ID:76/Command/General/SlaveZoomMode:1** - Enable slave zoom mode.

[system -> client] **ID:76/Ack** - Command accepted.

[client -> system] **ID:79/Query/General/VideoTrackerToken** - Query video tracker license token.

[system -> client]

ID:79/Reply/General/VideoTrackerToken:C7018FFD1BBE58C96DDD2534FF601FC62A112846FD13C30E5D7D5084B98C70D5A3FA53A96DA3D0B8B0404C6A98 - Video tracker license token.

4 7 ProcedureManager control messages

ProcedureManager is a submodule which is part of main board. **ProcedureManager** submodule is responsible for running scripts.

Table 4.7.1 - Action commands and commands to change parameters. Messages start from **.../Command/ProcedureManager/...** Response from system: **Ack**. Expected response time: **IP ping time to the system + 20 msec**.

Command	Description
GoToPreset:n	Go to preset. Parameter n (integer): preset ID [0:65535].

Some example messages:

[client -> system] **ID:82/Command/ProcedureManager/GoToPreset:2** - Go to preset 2.

[system -> client] **ID:82/Ack** - Command accepted.

4 8 LaserIlluminator control messages

LaserIlluminator is part of LightService submodule, but has its own communication channel.

LaserIlluminator submodule represents LaserIlluminator. The system can have only one LaserIlluminator. LaserIlluminator may be connected either to main board or to payload.

Table 4.8.1 - Action commands and commands to change parameters. Messages start from **.../Command/G5Laser/...**. Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**. Central system service sends command to **G5Laser** service and replies **Ack** if the message was sent without checking if the message is valid.

Command	Description
MODE/ARM:MODE	Arm or disarm the laser. Note: Laser must be armed before turning the actual light on. Parameter MODE (string): one of [OFF ON].
MODE/LIGHT:MODE	Set light output mode. Parameter MODE (string): one of [OFF BEAM STROBE]. Note: Strobe frequency is set by explicitly specifying duration of the light is on and the duration the time is off.
ARM-TIME:n	Set auto-disarm timeout in seconds. Parameter n (integer): time seconds [1-1000].
BEAM-TIME:n	Set beam (or strobe) auto-off timeout in seconds. Parameter n (integer): time seconds [1-1000].
STROBE-TIME/ON:n	Set strobe ON duration in milliseconds. Parameter n (integer): time milliseconds [1-1000].
STROBE-TIME/OFF:n	Set strobe OFF duration in milliseconds. Parameter n (integer): time milliseconds [1-1000].

Table 4.8.2 - Query commands. Messages start from **Query/G5Laser/...**. Response from system starts with **Reply/G5Laser/...**. Central control service sends message to **G5Laser** service. Expected response time: **IP ping time to the system + 5 msec** if central control service has received a valid reply from **G5Laser** service, or **IP ping time to the system + 105 msec** if no valid reply from **G5Laser** service.

Command	Description
READY	Query device readiness. Response: Reply/G5Laser/READY:STATE . Parameter STATE (string): one of [READY NOT READY].
VERSION	Query firmware version. Response: Reply/G5Laser/VERSION:VERSION . Parameter VERSION (string): firmware version string.
ARM/STATE	Query arm state. Response: Reply/G5Laser/ARM/STATE:STATE . Parameter STATE (string): one of [ON OFF].
ARM/TIME	Query configured arm timeout (seconds). Response: Reply/G5Laser/ARM/TIME:n . Parameter n (integer): time seconds.
ARM/TIMER	Query remaining arm time. Response: Reply/G5Laser/ARM/TIMER:TIME . Parameter TIME (string): remaining time seconds or DISARMED .

Command	Description
LIGHT/STATE	Query light mode. Response: Reply/G5Laser/LIGHT/STATE:MODE . Parameter MODE (string): one of [OFF BEAM STROBE].
LIGHT/TIME	Query configured beam timeout (seconds). Response: Reply/G5Laser/LIGHT/TIME:n . Parameter n (integer): time seconds.
LIGHT/TIME/STROBE/ON	Query strobe ON duration (ms). Response: Reply/G5Laser/LIGHT/TIME/STROBE/ON:n . Parameter n (integer): time milliseconds.
LIGHT/TIME/STROBE/OFF	Query strobe OFF duration (ms). Response: Reply/G5Laser/LIGHT/TIME/STROBE/OFF:n . Parameter n (integer): time milliseconds.
LIGHT/TIMER	Query remaining light time. Response: Reply/G5Laser/LIGHT/TIMER:TIME . Parameter TIME (string): remaining time seconds or OFF .
TEMP/LENS	Query lens temperature (Celsius). Response: Reply/G5Laser/TEMP/LENS:n . Parameter n (integer): temperature in Celsius.
TEMP/HOUSING	Query housing temperature (Celsius). Response: Reply/G5Laser/TEMP/HOUSING:n . Parameter n (integer): temperature in Celsius.

Some example messages:

[client -> system] **ID:90/Command/G5Laser/MODE/ARM:ON** - Arm the laser.

[system -> client] **ID:90/Ack** - Command accepted.

[client -> system] **ID:91/Command/G5Laser/MODE/LIGHT:BEAM** - Turn on light in BEAM mode.

[system -> client] **ID:91/Ack** - Command accepted.

[client -> system] **ID:92/Command/G5Laser/ARM-TIME:30** - Set auto-disarm timeout to 30 seconds.

[system -> client] **ID:92/Ack** - Command accepted.

[client -> system] **ID:93/Command/G5Laser/BEAM-TIME:60** - Set beam/strobe auto-off timeout to 60 seconds.

[system -> client] **ID:93/Ack** - Command accepted.

[client -> system] **ID:94/Command/G5Laser/STROBE-TIME/ON:100** - Set strobe ON duration to 100 ms.

[system -> client] **ID:94/Ack** - Command accepted.

[client -> system] **ID:95/Command/G5Laser/STROBE-TIME/OFF:50** - Set strobe OFF duration to 50 ms.

[system -> client] **ID:95/Ack** - Command accepted.

[client -> system] **ID:96/Query/G5Laser/READY** - Query device readiness.

[system -> client] **ID:96/Reply/G5Laser/READY:READY** - Device is ready.

[client -> system] **ID:97/Query/G5Laser/ARM/STATE** - Query arm state.

[system -> client] **ID:97/Reply/G5Laser/ARM/STATE:ON** - Laser is armed.

[client -> system] **ID:98/Query/G5Laser/LIGHT/STATE** - Query light mode.

[system -> client] **ID:98/Reply/G5Laser/LIGHT/STATE:BEAM** - Light mode is BEAM.

[client -> system] **ID:99/Query/G5Laser/TEMP/LENS** - Query lens temperature.

[system -> client] **ID:99/Reply/G5Laser/TEMP/LENS:32** - Lens temperature is 32C.

4 9 PeakBeam control messages

PeakBeam is part of LightService submodule, but has its own communication channel.

PeakBeam submodule represents PeakBeam. The system can have only one PeakBeam. PeakBeam may be connected either to main board or to payload.

Table 4.9.1 - Action commands and commands to change parameters. Messages start from **.../Command/PeakBeam/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
Mode:MODE	Set operating mode. Parameter MODE (string): one of [OFF BEAM STROBE TURBO].
Level:LEVEL	Set light intensity. Parameter LEVEL (string): one of [Low Normal High]. Note 1: turning light on always triggers the maximum power for 2 second to heat up. Note 2: power level doesn't affect strobe mode.
StrobeRate:n	Set strobe frequency in Hz. Parameter n (integer): rate [1-31]. Note: device additionally supports 50hz frequency, it activates via TURBO mode.
ZoomRate:n	Set zoom speed as percentage of slowest speed. Parameter n (integer): rate [1-100], where 100 is the slowest possible speed.
ToPos:n	Move zoom to absolute position. Parameter n (integer): position [0-99]. Note: ZoomRate doesn't affect the speed of ToPos command. It's always executed at maximum speed.
Zoom:DIR	Start continuous zoom movement. Parameter DIR (string): one of [In\Out].
ZoomStop	Stop zoom movement.

Table 4.9.2 - Query commands. Messages start from **Query/PeakBeam/....** Response from system starts with **Reply/PeakBeam/....** Central control service sends message to **PeakBeam** service. Expected response time: **IP ping time to the system + 5 msec** if central control service has received a valid reply from **PeakBeam** service, or **IP ping time to the system + 105 msec** if no valid reply from **PeakBeam** service.

Command	Description
Mode	Query current mode. Response: Reply/PeakBeam/Mode:MODE . Parameter MODE (string): one of [OFF BEAM STROBE].
Level	Query current level. Response: Reply/PeakBeam/Level:LEVEL . Parameter LEVEL (string): one of [Low Normal High].
StrobeRate/Pcnt	Query strobe rate as percentage. Response: Reply/PeakBeam/StrobeRate/Pcnt:n . Parameter n (integer): strobe rate.
StrobeRate/Hz	Query strobe rate in Hz. Response: Reply/PeakBeam/StrobeRate/Hz:n . Parameter n (integer): rate Hz.
ZoomRate/Pcnt	Query zoom speed as percentage. Response: Reply/PeakBeam/ZoomRate/Pcnt:n . Parameter n (integer): zoom rate percent.

Command	Description
ZoomRate/Raw	Query zoom speed raw value. Response: Reply/PeakBeam/ZoomRate/Raw:n . Parameter n (integer): raw zoom rate.
Pos	Query current zoom position. Response: Reply/PeakBeam/Pos:n . Parameter n (integer): position [0-99].
Temperature	Query housing temperature in Celsius. Response: Reply/PeakBeam/Temperature:n . Parameter n (integer): temperature in Celsius.
Version	Query firmware version. Response: Reply/PeakBeam/Version:VERSION . Parameter VERSION (string): firmware version string.
ZoomState	Query zoom state. Response: Reply/PeakBeam/ZoomState:STATE . Parameter STATE (string): one of [NONE In Out GO_TO].

Some example messages:

[client -> system] **ID:100/Command/PeakBeam/Mode:BEAM** - Set operating mode BEAM.
[system -> client] **ID:100/Ack** - Command accepted.
[client -> system] **ID:101/Command/PeakBeam/Level:High** - Set light intensity to High.
[system -> client] **ID:101/Ack** - Command accepted.
[client -> system] **ID:102/Command/PeakBeam/StrobeRate:15** - Set strobe frequency to 15 Hz.
[system -> client] **ID:102/Ack** - Command accepted.
[client -> system] **ID:103/Command/PeakBeam/ZoomRate:50** - Set zoom speed to 50%.
[system -> client] **ID:103/Ack** - Command accepted.
[client -> system] **ID:104/Command/PeakBeam/ToPos:75** - Move zoom to position 75.
[system -> client] **ID:104/Ack** - Command accepted.
[client -> system] **ID:105/Command/PeakBeam/Zoom:In** - Start continuous zoom in.
[system -> client] **ID:105/Ack** - Command accepted.
[client -> system] **ID:106/Command/PeakBeam/ZoomStop** - Stop zoom movement.
[system -> client] **ID:106/Ack** - Command accepted.
[client -> system] **ID:107/Query/PeakBeam/Mode** - Query current mode.
[system -> client] **ID:107/Reply/PeakBeam/Mode:BEAM** - Current mode is BEAM.
[client -> system] **ID:108/Query/PeakBeam/StrobeRate/Hz** - Query strobe rate in Hz.
[system -> client] **ID:108/Reply/PeakBeam/StrobeRate/Hz:15** - Strobe rate is 15 Hz.
[client -> system] **ID:109/Query/PeakBeam/Pos** - Query zoom position.
[system -> client] **ID:109/Reply/PeakBeam/Pos:75** - Zoom position is 75.
[client -> system] **ID:110/Query/PeakBeam/Temperature** - Query housing temperature.
[system -> client] **ID:110/Reply/PeakBeam/Temperature:32** - Housing temperature is 32C.

4 10 ExternalUp control messages

ExternalUp is a submodule which is part of main board. **ExternalUp** submodule is responsible for external Universal Protocol communication parameters control.

Table 4.10.1 - Action commands and commands to change parameters. Messages start from **.../Command/ExternalUp/....** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec**.

Command	Description
UdpPort:n	Set external Universal Protocol UDP control port. Parameter n (integer): UDP port [500-65535]. WARNING: after parameter change the ExternalUpService restart or system restart required.
TcpPort:n	Set external Universal Protocol TCP control port. Parameter n (integer): TCP port [500-65535]. WARNING: after parameter change the ExternalUpService restart or system restart required.

Table 4.10.2 - ExternalUp submodule queries. Messages start from **Query/ExternalUp/...** Response from system starts with **Reply/ExternalUp/...** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
UdpPort	Query external Universal Protocol UDP control port. Response: Reply/ExternalUp/UdpPort:n . Parameter n (integer): UDP port [500-65535].
TcpPort	Query external Universal Protocol TCP control port. Response: Reply/ExternalUp/TcpPort:n . Parameter n (integer): TCP port [500-65535].

Some example messages:

[client -> system] **ID:106/Command/ExternalUp/UdpPort:34020** - Set UDP port to 34020.

[system -> client] **ID:106/Ack** - Command accepted.

[client -> system] **ID:107/Query/ExternalUp/UdpPort** - Query UDP port.

[system -> client] **ID:107/Reply/ExternalUp/UdpPort:34020** - Current UDP port is 34020.

[client -> system] **ID:108/Query/ExternalUp/TcpPort** - Query TCP port.

[system -> client] **ID:108/Reply/ExternalUp/TcpPort:34030** - Current TCP port is 34030.

4 11 PelcoD control messages

PelcoD is a submodule which is part of main board. **PelcoD** submodule is responsible for external PelcoD communication parameters control.

Table 4.11.1 - Action commands and commands to change parameters. Messages start from **.../Command/PelcoD/...** Response from system: **Ack**. Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
SerialBaudrate:n	PelcoD serial port baudrate. Parameter n (integer) valid values: 9600, 19200, 38400, 57600, 115200. WARNING: after parameter change the PelcoDService restart or system restart required.
UdpPort:n	Set external PelcoD UDP control port. Parameter n (integer): UDP port [500-65535]. WARNING: after parameter change the PelcoDService restart or system restart required.
TcpPort:n	Set external PelcoD TCP control port. Parameter n (integer): TCP port [500-65535]. WARNING: after parameter change the PelcoDService restart or system restart required.

Table 4.11.2 - PelcoD submodule queries. Messages start from **Query/PelcoD/....** Response from system starts with **Reply/PelcoD/....** Expected response time: **IP ping time to the system + 5 msec.**

Command	Description
SerialBaudrate	Query PelcoD serial port baudrate. Response: Reply/PelcoD/SerialBaudrate:n . Parameter n (integer) valid values: 9600, 19200, 38400, 57600, 115200.
SerialsOpen	Query PelcoD serial port status. Response: Reply/PelcoD/SerialsOpen:n . Parameter n (integer): 0 - not open (no communication), 1 - open (ready for communication).
UdpPort	Query PelcoD UDP control port. Response: Reply/PelcoD/UdpPort:n . Parameter n (integer): UDP port [500-65535].
TcpPort	Query PelcoD TCP control port. Response: Reply/PelcoD/TcpPort:n . Parameter n (integer): TCP port [500-65535].

Some example messages:

```
[client -> system] ID:106/Command/PelcoD/UdpPort:34000 - Set UDP port to 34000.
[system -> client] ID:106/Ack - Command accepted.
[client -> system] ID:107/Query/PelcoD/UdpPort - Query UDP port.
[system -> client] ID:107/Reply/PelcoD/UdpPort:34000 - Current UDP port is 34000.
[client -> system] ID:108/Query/PelcoD/TcpPort - Query TCP port.
[system -> client] ID:108/Reply/PelcoD/TcpPort:34010 - Current TCP port is 34010.
```

4 12 Combined commands

The **NexOS** system support combined commands. Combined commands are commands which combined multiple commands in one. User need put end of line symbol '\n' between commands. Example:

ID:82/Command/ProcedureManager/GoToPreset:2\nID:83/Command/Camera1/Lens/ZoomIn\nID:84/Query/Camera1/Lens/FocusPos.

The NexOS system will reply to each command included in combined command: Example:

Client -> system:

ID:82/Command/ProcedureManager/GoToPreset:2\nID:83/Command/Camera1/Lens/ZoomIn\nID:84/Query/Camera1/Lens/FocusPos

System -> client:

ID:82/Ack

ID:83/Ack

ID:84/Reply/Camera1/Lens/FocusPos:10000

5 Example of control program

Below is source code of test application which uses **UdpSocket** C++ class to work with UDP sockets. This application is used to send any command to **JafarService** and check responses. Also, this application measures response time. The example is represented by source code of two files: **CMakeLists.txt** and **main.cpp**. The application doesn't have third-party dependencies and compatible with **linux** and **Windows**. To compile application in Linux the user has to install necessary build packets (example for Ubuntu):

```
sudo apt-get update
sudo apt-get install cmake build-essential nano
```

After user has to create **CMakeLists.txt** and **main.cpp**. Commands to create files and build application in Linux:

```
cd <your folder>
nano CMakeLists.txt
<copy content listed below for CMakeLists.txt there and press Ctrl+S and then Ctrl+X to
close editor>
nano main.cpp
<copy content listed below for main.cpp there and press Ctrl+S and then Ctrl+X to close
editor>
mkdir build
cd build
cmake ..
make
```

To run application use command:

```
cd bin
./EupControlApplication
```

Content of **CMakeLists.txt** file:

```
cmake_minimum_required(VERSION 3.13)

# Project name
project(EupControlApplication LANGUAGES CXX)

# Settings
set(CMAKE_INCLUDE_CURRENT_DIR ON)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Creating output directory architecture
set(BINARY_DIR "${CMAKE_BINARY_DIR}")
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY "${BINARY_DIR}/bin")

# Create glob files for *.h, *.cpp
file(GLOB H_FILES ${CMAKE_CURRENT_SOURCE_DIR}/*.h)
file(GLOB CPP_FILES ${CMAKE_CURRENT_SOURCE_DIR}/*.cpp)

# Concatenate the results (glob files) to variable
set(SOURCES ${CPP_FILES} ${H_FILES})
if(NOT TARGET ${PROJECT_NAME})
    add_executable(${PROJECT_NAME} ${SOURCES})
endif()

# Link network libraries for windows
if(WIN32)
    target_link_libraries(${PROJECT_NAME} ws2_32)
    if(MINGW)
```

```
        target_link_libraries(${PROJECT_NAME} iphlpapi)
    endif(MINGW)
endif()
```

Content of **main.cpp** file:

```
#include <iostream>
#include <thread>
#include <chrono>
#include <string>
#include <cstring>
#include <cstdio>
#include <stdint>
#include <atomic>
#if defined(linux) || defined(__linux) || defined(__linux__)
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#else
#define WIN32_LEAN_AND_MEAN
#include <winsock2.h>
#include <ws2tcpip.h>
#endif

// Link namespaces.
using namespace std;
using namespace std::chrono;

/**
 * @brief UDP socket class.
 */
class UdpSocket
{
public:

    /**
     * @brief Class constructor.
     */
    UdpSocket();

    /**
     * @brief Class destructor.
     */
    ~UdpSocket();

    /**
     * @brief Open UDP socket.
     * @param port UDP port.
     * @param serverType TRUE to send/receive data, FALSE only to send data.
     * (socket will not be bind).
     * @param timeoutMsec Wait data timeout in milliseconds.
     * @return TRUE in socket open or FALSE if not.
     */
    bool Open(int port, bool serverType, int timeoutMsec);
```

```

bool open(uint16_t port, bool serverType = false,
          string dstIp = "127.0.0.1", int timeoutMsec = 100);

/**
 * @brief Read data.
 * @param data Pointer to data buffer to copy data (not nullptr).
 * @param size Size of buffer.
 * @param srcAddr Pointer to structure from which the data was read.
 * @return Number of read bytes or return -1 in case error.
 */
int read(uint8_t* data,
         int size,
         sockaddr_in* srcAddr = nullptr);

/**
 * @brief Send data.
 * @param data Pointer to data to send.
 * @param size Size of data to send.
 * @param dstAddr Pointer to structure to data to send.
 * @return Number of bytes sent or return -1 if UDP socket not open.
 */
int send(uint8_t* data, int size, sockaddr_in* dstAddr = nullptr);

/**
 * @brief Check if UDP socket open.
 * @return TRUE if socket open or FALSE if not.
 */
bool isOpen();

/**
 * @brief Close UDP socket.
 */
void close();

/**
 * @brief Get IP of data source.
 * @param srcAddr Pointer to structure from which the data was read.
 * @return IP of data source.
 */
string getIp(sockaddr_in* srcAddr);

/**
 * @brief Get UDP port of data source.
 * @param srcAddr Pointer to structure from which the data was read.
 * @return UDP port of data source.
 */
int getPort(sockaddr_in* srcAddr);

private:

// Socket open flag.
atomic<bool> m_isOpen{false};
// Destination net address structure.
struct sockaddr_in m_dstAddr{0};
// Source net address structure.
struct sockaddr_in m_srcAddr{0};

```

```

// Socket.
#ifdef __linux__ || defined(__linux) || defined(__linux__)
    int m_sock{0};
#else
    SOCKET m_sock{0};
#endif
};

/// Entry point.
int main(void)
{
    // Set system IP.
    string ip = "192.168.1.86";
    cout << "System IP: ";
    cin >> ip;

    // Open UDP port with 500 msec timeout (wait data timeout).
    uint16_t port = 34020; // Default port for external Up control.
    UdpSocket udpSocket;
    int timeoutMsec = 500;
    if (!udpSocket.open(port, true, ip, timeoutMsec))
    {
        cout << "UDP port " << port << " not open" << endl;
        this_thread::sleep_for(seconds(1));
        return -1;
    }
    cout << "UDP port " << port << " open" << endl;

    // Main loop.
    int id = 0;
    while (true)
    {
        // Clean buffer just in case to not receive previous messages.
        uint8_t response[2048];
        while (udpSocket.read(response, 2048) > 0)
            udpSocket.read(response, 2048);

        // Enter message (Command or Query).
        cout << "-----" << endl;
        string command;
        cout << "Message (starts with Command or Query): ";
        cin >> command;
        command = "ID:" + to_string(id++) + "/" + command;

        // Send command.
        time_point<system_clock> start = system_clock::now();
        if (udpSocket.send((uint8_t*)command.data(), command.size()) < 0)
        {
            cout << "Can't send command" << endl;
            continue;
        }

        // Receive response.
        int size = udpSocket.read(response, sizeof(response));
    }
}

```

```

        if (size <= 0)
        {
            cout << "No response in " << timeoutMsec << " msec" << endl;
            continue;
        }
        int responseTimeMsec =
        (int)duration_cast<milliseconds>(system_clock::now() - start).count();

        // Show response.
        cout << "Response " << responseTimeMsec << " msec : " <<
        string(response, response + size) << endl;
    }

    return 1;
}

UdpSocket::UdpSocket()
{
    // Reset address structures.
    memset(&m_dstAddr, 0, sizeof(sockaddr_in));
    memset(&m_srcAddr, 0, sizeof(sockaddr_in));
}

UdpSocket::~UdpSocket()
{
    // Close socket.
    close();
}

bool UdpSocket::open(uint16_t port, bool serverType, string dstIp, int timeoutMsec)
{
    // Close socket first.
    close();

    // Init params in windows OS.
    #if defined(linux) || defined(__linux) || defined(__linux__)
    #else
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0)
        return false;
    #endif

    // Init socket.
    m_sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    // Close socket in case fail initialization
    #if defined(linux) || defined(__linux) || defined(__linux__)
    if (m_sock == -1)
    {
        #else

```

```

if (m_sock == INVALID_SOCKET)
{
    WSACleanup();
#endif
    return false;
}

// Bind socket if server type.
if (serverType)
{
    struct sockaddr_in hostAddr;
    memset(&hostAddr, 0, sizeof(sockaddr_in));
    hostAddr.sin_family = AF_INET;
    string ip = "0.0.0.0";
    if (!inet_pton(AF_INET, ip.c_str(), &hostAddr.sin_addr))
        return false;
    hostAddr.sin_port = htons(port);

#ifdef linux || defined(__linux) || defined(__linux__)
    if (::bind(m_sock, (struct sockaddr*)&hostAddr, sizeof(hostAddr)) < 0)
    {
#else
    if (::bind(m_sock, (SOCKADDR*)&hostAddr, sizeof(hostAddr)) ==
        SOCKET_ERROR) {
#endif
        // Close socket in case fail bind.
#ifdef linux || defined(__linux) || defined(__linux__)
        ::close(m_sock);
#else
        closesocket(m_sock);
        WSACleanup();
#endif
        return false;
    }
}

#ifdef linux || defined(__linux) || defined(__linux__)
// Init timeouts
timeval timeparams;
timeparams.tv_sec = timeoutMsec / 1000;
// Timeout in microseconds for read data from socket.
timeparams.tv_usec = timeoutMsec % 1000;
if (timeoutMsec != 0)
{
    // Close socket in case error
    if (setsockopt(m_sock, SOL_SOCKET, SO_RCVTIMEO,
        (const char*)&timeparams, sizeof(timeval)) < 0)
    {
        ::close(m_sock);
    }
}
#else
// Init timeouts
// Timeout in milliseconds for read data from socket.
DWORD _timeout = timeoutMsec;
if (_timeout != 0)
{
    // Close socket in case error

```

```

        if (setsockopt(m_sock, SOL_SOCKET, SO_RCVTIMEO,
            (const char*)&_timeout, sizeof(_timeout)) < 0)
        {
            closesocket(m_sock);
            WSACleanup();
        }
    #endif
    return false;
}

#if defined(linux) || defined(__linux) || defined(__linux__)
int trueflag = 1;
if (setsockopt(m_sock, SOL_SOCKET, SO_BROADCAST,
    &trueflag, sizeof(trueflag)) < 0)
{
    ::close(m_sock);
}
#else
const char trueflag = 1;
// Close socket in case error
if (setsockopt(m_sock, SOL_SOCKET, SO_BROADCAST,
    &trueflag, sizeof(trueflag)) < 0)
{
    closesocket(m_sock);
    WSACleanup();
}
#endif
return false;
}

// Set dst address.
memset(&m_dstAddr, 0, sizeof(sockaddr_in));
m_dstAddr.sin_family = AF_INET;
if (!inet_pton(AF_INET, dstIp.c_str(), &m_dstAddr.sin_addr)) {
    close();
    return false;
}
m_dstAddr.sin_port = htons(port);

// Init flag
m_isOpen = true;

return true;
}

int UdpSocket::read(uint8_t* data, int size, sockaddr_in* srcAddr)
{
    // Check if socket not open.
    if (!m_isOpen)
        return -1;

    // wait and read data from socket.
    int ret = 0;
    #if defined(linux) || defined(__linux) || defined(__linux__)
    socklen_t _srcAddrSize = sizeof(m_srcAddr);
    ret = recvfrom(m_sock, reinterpret_cast<char*>(data), size, 0,
        (struct sockaddr*)&m_srcAddr, &_srcAddrSize);
    #else

```

```

    int _srcAddrSize = sizeof(m_srcAddr);
    ret = recvfrom(m_sock, reinterpret_cast<char*>(data), size, 0,
                  (struct sockaddr*)&m_srcAddr, &_srcAddrSize);
#endif

    if (srcAddr != nullptr)
        memcpy(srcAddr, &m_srcAddr, _srcAddrSize);

    return ret;
}

int UdpSocket::send(uint8_t* data, int size, sockaddr_in* dstAddr)
{
    // Check if socket not open.
    if (!m_isOpen)
        return -1;

    // Send data.
    if (dstAddr != nullptr)
        return sendto(m_sock, reinterpret_cast<char*>(data), size, 0,
                      (struct sockaddr*)dstAddr, sizeof(sockaddr_in));
    else
        return sendto(m_sock, reinterpret_cast<char*>(data), size, 0,
                      (struct sockaddr*)&m_dstAddr, sizeof(sockaddr_in));
}

bool UdpSocket::isOpen() {
    return m_isOpen;
}

void UdpSocket::close()
{
    // close socket
    if (m_isOpen)
    {
#ifdef defined(linux) || defined(__linux) || defined(__linux__)
        ::close(m_sock);
    #else
        closesocket(m_sock);
        WSACleanup();
    #endif
    }
    m_isOpen = false;
}

string UdpSocket::getIp(sockaddr_in* srcAddr)
{
    char str[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &(srcAddr->sin_addr), str, INET_ADDRSTRLEN);
    return string(str);
}

int UdpSocket::getPort(sockaddr_in* srcAddr)
{
    return htons(srcAddr->sin_port);
}

```

6 Example of detection results subscription

To receive messages about detected objects, you must register a UDP socket in the system. The example shows how to initialize the socket, send subscription commands and how to receive data from the system (**UdpSocket** class was explained in previous [example](#)). To receive detection results and send **Subscribe** command use separate UDP socket but not UDP socket to send control commands.

```
#include <iostream>
#include <string>
#include <thread>
#include <chrono>
#include <fstream>
#include <vector>
#include "UdpSocket.h"
#include "TcpClient.h"

// Link namespaces.
using namespace std;
using namespace cr::clib;
using namespace std::chrono;

/// Entry point.
int main(void)
{
    // Set system IP.
    string ip = "192.168.1.80";
    cout << "System IP: ";
    cin >> ip;

    // Set UDP port.
    int port = 34020;
    cout << "UDP port: ";
    cin >> port;

    // Open UDP port with 500 msec timeout (wait data timeout).
    UdpSocket udpSocket;
    int timeoutMsec = 500;
    if (!udpSocket.open(port, true, ip, timeoutMsec))
    {
        cout << "UDP port " << port << " not open" << endl;
        this_thread::sleep_for(seconds(1));
        return -1;
    }
    cout << "UDP port " << port << " open" << endl;

    // Set command.
    //cout << "Command (example: Camera1/Classification/Objects/Subscribe): ";
    string command = "Camera1/Classification/Objects/Subscribe";
    cin >> command;
    command = "ID:0/Command/" + command + "\n";

    // Send Subscribe for MotionDetector of Camera1.
    if (udpSocket.send((uint8_t*)command.data(), command.size()) < 0)
```

```

{
    cout << "Can't send subscription" << endl;
    return -1;
}
cout << "TX: " << command << endl;

// Loop to wait messages.
uint8_t data[2048];
while (true)
{
    // Read data.
    int size = udpSocket.read(data, sizeof(data));
    if (size <= 0)
    {
        cout << "No data in " << timeoutMsec << " msec" << endl;
        continue;
    }

    // Show data.
    cout << "RX: " << string(data, data + size) << endl;
}
}

```

7 JSON based communication

NexOS allows messages to be exchanged in JSON format. The message is exchanged on the same port as EUP messages. The **NexOS** system recognizes which type of message comes and processes it according to format. If **NexOS** system receives string-formatted message (**UP** format) it will send back to client string-formatted reply. If **NexOS** system receives JSON-formatted message (**UP** format) it will send back to client JSON-formatted reply. There are direct conversions between string-based UP messages. Some examples of JSON based equivalent to UP messages:

UP string-based command (from client to system):

```
ID:0/Command/Camera1/Lens/ZoomIn
```

Equal to UP JSON-based command:

```

{
    "ID": "0",
    "To": "Camera1",
    "Type": "Command",
    "Content": ["Lens", "ZoomIn"]
}

```

UP string-based replay (from system to client):

```
ID:0/Ack
```

Equal to UP JSON-based reply:

```
{
  "ID": "0",
  "Type": "Ack"
}
```

UP string-based command (from client to system):

```
ID:1/Command/Lrf/MinGateDistance:10
```

Equal to UP JSON-based command:

```
{
  "ID": "1",
  "To": "Lrf",
  "Type": "Command",
  "Content": ["Lrf", "MinGateDistance:10"]
}
```

UP string-based reply (from system to client)

```
ID:1/Ack
```

Equal to UP JSON-based reply:

```
{
  "ID": "1",
  "Type": "Ack"
}
```

UP string-based query (from client to system):

```
ID:2/Query/Camera2/MotionDetector/Mode
```

Equal to UP JSON-based query:

```
{
  "ID": "2",
  "To": "Camera2",
  "Type": "Query",
  "Content": ["MotionDetector", "Mode"]
}
```

UP string-based reply (from system to client):

```
ID:2/Reply/Camera2/MotionDetector/Mode:1
```

Equal to UP JSON-based reply:

```
{
  "ID": "2",
  "From": "Camera2",
  "Type": "Reply",
  "Content": ["MotionDetector", "Mode:1"]
}
```

Example of sending JSON-based commands and receiving JSON-based responses:

```
int main(void)
{
    // Set system IP.
    string ip = "192.168.1.86";
    cout << "System IP: ";
    cin >> ip;

    // Open UDP port with 500 msec timeout (wait data timeout).
    uint16_t port = 34020; // Default port for external Up control.
    UdpSocket udpSocket;
    int timeoutMsec = 500;
    if (!udpSocket.open(port, true, ip, timeoutMsec))
    {
        cout << "UDP port " << port << " not open" << endl;
        this_thread::sleep_for(seconds(1));
        return -1;
    }
    cout << "UDP port " << port << " open" << endl;

    // Main loop.
    int id = 0;
    while (true)
    {
        // Clean buffer just in case to not receive previous messages.
        uint8_t response[2048];
        while (udpSocket.read(response, 2048) > 0)
            udpSocket.read(response, 2048);

        cout << "-----" << endl;

        string Type = "Command";
        cout << "Type (1 - Command or 2 - Query): ";
        cin >> Type;
        if (Type == "1")
            Type = "Command";
        else
            Type = "Query";

        string To = "Camera1";
        cout << "Destination: ";
        cin >> To;

        // Enter list of strings.
        vector<string> Content;
        string line;
        cout << "Enter strings separated by spaces: ";
```

```

cin.ignore(); // Ignore the newline character left in the buffer
getline(cin, line);
stringstream ss(line);
string arg;
while (ss >> arg)
    Content.push_back(arg);

// Create message.
nlohmann::json j;
j["ID"] = to_string(id++);
j["To"] = To;
j["Type"] = Type;
j["Content"] = Content;

// Send command.
string command = j.dump();
time_point<system_clock> start = system_clock::now();
udpSocket.send((uint8_t*)command.data(), command.size());

// Show sent message.
cout << "-----" << endl;
cout << "[TX] " << command << endl;

// Receive response.
int size = udpSocket.read(response, sizeof(response));
if (size <= 0)
{
    cout << "No response in " << timeoutMsec << " msec" << endl;
    continue;
}
int responseTimeMsec =
(int)duration_cast<milliseconds>(system_clock::now() - start).count();

// Show RAW message.
cout << "[RX] " << string(response, response + size) << endl;

// Parse JSON message.
nlohmann::json inJ = nlohmann::json::parse(response, response + size);
string id = "None";
if (!inJ["ID"].is_null())
    id = inJ["ID"];
string from = "None";
if (!inJ["From"].is_null())
    from = inJ["From"];
string type = "None";
if (!inJ["Type"].is_null())
    type = inJ["Type"];
vector<string> content;
if (!inJ["Content"].is_null())
    content = inJ["Content"].get<vector<string>>();

// Show parsed message.
cout << "ID: " << id << endl;
cout << "From: " << from << endl;
cout << "Type: " << type << endl;
cout << "Content: ";

```

```
    for (auto& c : content)
        cout << c << " ";
    cout << endl;
}

return 1;
}
```